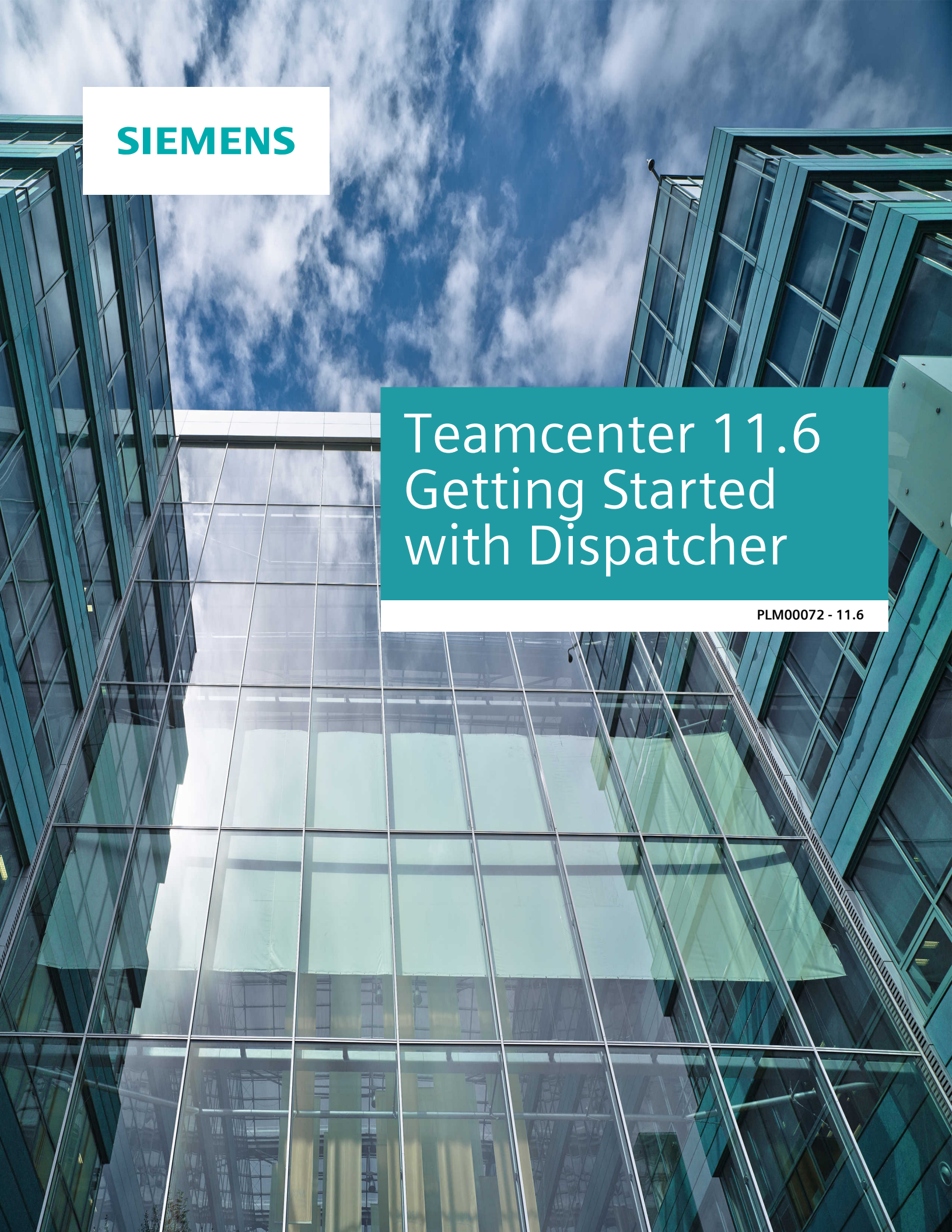


SIEMENS

Teamcenter 11.6 Getting Started with Dispatcher

PLM00072 - 11.6

Contents

Customizing Dispatcher

Troubleshooting

Service.properties file reference

1. Overview of Dispatcher

Overview of Dispatcher

Dispatcher is the integration of the Dispatcher Server and Teamcenter. Dispatcher enables Teamcenter users to translate data files that are managed by Teamcenter to 3D or 2D file formats.

Before you begin

Prerequisites	Before installing Dispatcher, you must ensure you have the required hardware and software. This section describes the hardware and software you require to install Dispatcher. **Hardware requirements** Siemens PLM Software recommends that you install translation modules on enterprise-class servers only. Enterprise-class servers are better designed to handle the computational, memory, and multithreaded requirements of translators for large models. **Software requirements** • Dispatcher has the same software requirements as Teamcenter. For information about versions of operating systems, third-party software, and Teamcenter configurations that are certified for your platform, see the **Siemens PLM Software Certification Database**.
Enable Dispatcher	To enable Dispatcher: • Install the CAD design tools. • Install Teamcenter. • Install the Dispatcher feature. • Set up translators.
Configure Dispatcher	The Dispatcher components are configured automatically when you install Dispatcher using TEM. After installation, you must: • Create Dispatcher access rules. • Start Dispatcher services.

Dispatcher interface

Dispatcher interface

Dispatcher uses the Teamcenter rich client interface.

Dispatcher menus

All Dispatcher menus are standard Teamcenter rich client menus except for those described here.

Command	Description
Translation→Translate...	Starts translation of the selected dataset.
Translation→Administrator Console – All	Opens the Request Administration Console.

Dispatcher request administration console interface

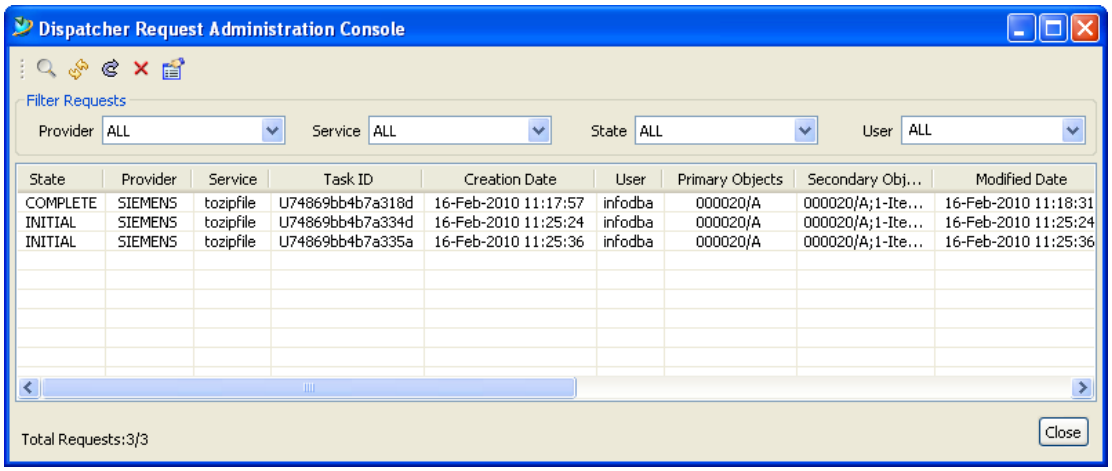


Figure 1-1. Dispatcher request administration console interface

Dispatcher request administration console buttons

Buttons	Description
	Refreshes all dispatcher requests.
	Refreshes the selected dispatcher request.
	Resubmits a dispatcher request for processing.

Buttons	Description
	Deletes the selected dispatcher request.
	Launches the properties dialog box for the selected dispatcher request.

Teamcenter rich client perspectives and views

Within the Teamcenter rich client user interface, functionality is provided in *perspectives* and *views*. Some applications use perspectives and views to rearrange how the functionality is presented. Other applications use a single perspective and view to present information.

- Perspectives
  Are containers for a set of views and editors that exist within the perspective.
  - A perspective exists in a window along with any number of other perspectives, but only one perspective can be displayed at a time.
  - In applications that use multiple views, you can add and rearrange views to display multiple sets of information simultaneously within a perspective.
  - You can save a rearranged perspective with the current name, or create a new perspective by saving the new arrangement of views with a new name.
- Views and view networks
  In some Teamcenter applications, using rich client views and view networks, you can navigate to a hierarchy of information, display information about selected objects, open an editor, or display properties.
  - Views that work with related information typically react to selection changes in other views.
  - Changes to data made in a view can be saved immediately.
  - Any view can be opened in any perspective, and any combination of views can be saved in a current perspective or in a new perspective.
  - A view network consists of a primary view and one or more secondary views that are associated. View networks can be arranged in a single view folder or in multiple view folders.
  - Objects selected in a view may provide context for a shortcut menu. The shortcut menu is usually displayed by right-clicking.

Note:

If your site has online help installed, you can access application and view help from the rich client **Help** menu or by pressing F1. Some views, such as **Communication Monitor**, **Print Object**, and **Performance Monitor**, are auxiliary views that may be used for debugging and that may not be displayed automatically by any particular perspective.

Basic concepts about Dispatcher

Basic concepts about Dispatcher

Dispatcher is the integration of the Dispatcher Server and Teamcenter. Dispatcher enables Teamcenter users to manage job distribution and execution.

With Dispatcher, you can:

- Asynchronously distribute jobs to machines with the resource capacity to execute the job. (The jobs can also be done in a synchronous manner on the local client machines, but this can introduce performance and administration issues based on the job load.)
- Reduce server load by distributing resource-intensive activities.
- Schedule high-CPU or high-memory jobs for off-hours processing.
- Get a grid technology solution to manage the job distribution, communication, execution, security, and error handling. You can scale and customize the Dispatcher system to meet specific site requirements.

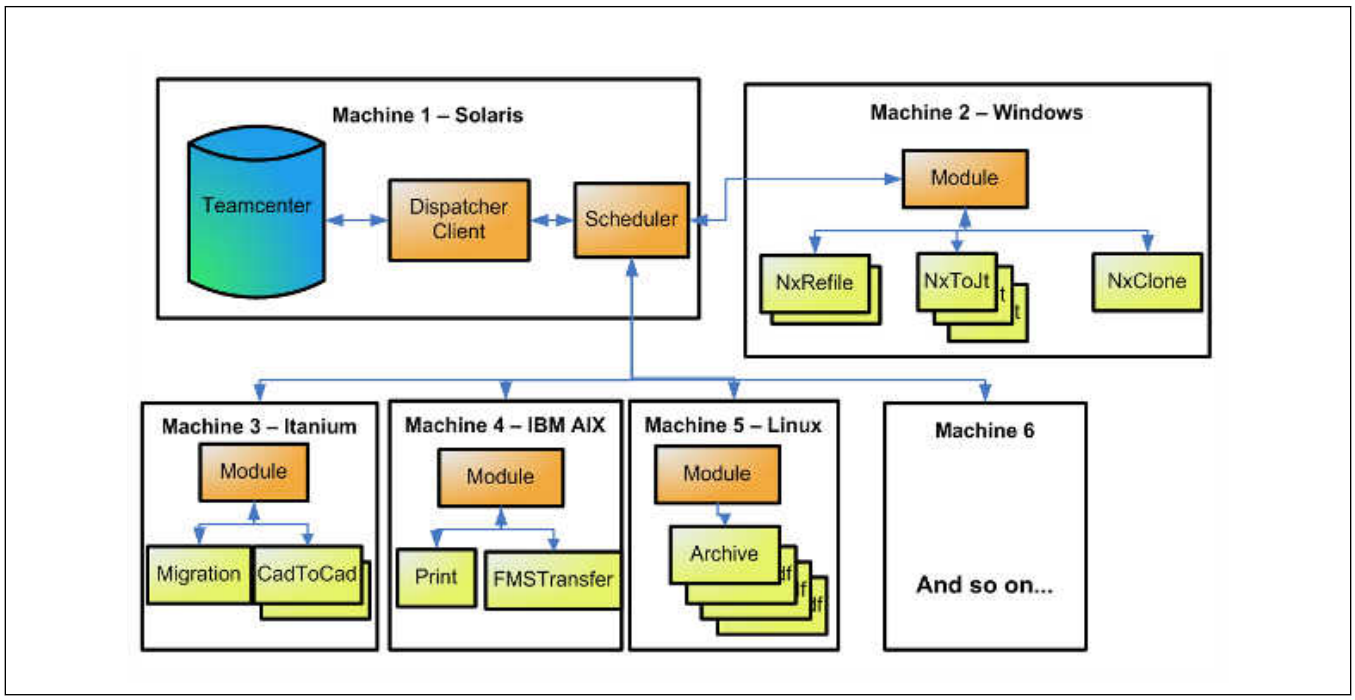
Dispatcher is mostly used to translate data files that are managed by Teamcenter to 3D or 2D file formats. You can:

- View, edit, and mark up data without requiring the original data's authoring tool.
- Share the data with other companies without distributing the original raw data (CAD, MS Office, and so forth).
- Generate a lightweight data format known as visualization data. The visualization data can be attached along with the raw data and can be viewed by Teamcenter viewers or by third-party applications capable of reading and displaying the format.

Dispatcher components

Dispatcher components

Figure shows the components of a Dispatcher system. The scheduler, module, and translators are Dispatcher Server components.



Dispatcher components

Dispatcher client

The *dispatcher client* provides the framework to extract and load data to Teamcenter. The dispatcher client provides the communication mechanism to use the Dispatcher Server components.

The dispatcher client communicates with the Dispatcher Server using Remote Method Invocation (RMI) or Hypertext Transfer Protocol (HTTP). RMI is the default. Using HTTP causes file transfer performance issues and it needs more installation steps. The dispatcher client uses Service Oriented Architecture (SOA) to communicate with the server.

The dispatcher client is only supported on a two-tier version of Teamcenter.

Scheduler

The *scheduler* manages distribution of translation requests across modules. Translation tasks are submitted to the scheduler by Teamcenter via RMI or HTTP communication. The scheduler tracks what modules are available and the translators each module has available. It uses this information to distribute the translation tasks to keep each module running to its maximum capacity.

The scheduler is a Java™-based RMI application. It comprises exposed interfaces used by the dispatcher client to send and receive information about dispatcher requests. The scheduler communicates with the module using RMI.

Module

The *module* is used for dispatching the specific translators required for translation tasks. It provides the infrastructure for a common way to plug in and execute any translator and support the various command line options, parameters, and configuration files unique to each translator.

The module is a Java-based RMI application. It has exposed RMI interfaces used by the scheduler to send and receive information about translation requests.

Dispatcher request administration console

The Dispatcher request administration console allows you to manage and monitor the status of a task submitted for translation. With the Dispatcher request administration console, you can:

- See all translation requests.
- Resubmit a translation request for processing.
- Delete a translation request.

DispatcherRequest object

DispatcherRequest object

The Dispatcher task execution process starts by the creation of a **DispatcherRequest** object. The **DispatcherRequest** object is represented by the **DispatcherRequest** class.

DispatcherRequest object attributes

Attribute	Description
Current state	Displays the current state of the dispatcher request.
Provider name	Displays the name of the provider of the translator.
Service name	Displays the name of the dispatcher service to be used.
Priority	Specifies a scheduling priority. Set it to low, medium, or high.
Task ID	Displays the task identifier assigned during the preparation process.
Creation date	Displays the date and time when the **DispatcherRequest** object was created.

Attribute	Description
Primary objects	Specifies one or more primary objects containing the data to be translated. When translating CAD data, primary objects are typically datasets.
Secondary objects	Specifies one or more supporting objects for the translation. Note: Secondary objects are optional and dependant on the requirements of the specific translation. When translating CAD data, secondary objects are typically item revisions.
Owning user	Displays the name of the user that created the request.
Owning group	Displays the name of the group that created the request.
History states	Displays various states that make up the history of the dispatcher request.
History dates	Displays timestamps associated with state values.
Type	Shows an application the purpose of the dispatcher request.
Mode	Represents a user-defined mode of operation.
Argument keys/argument data	Specifies one or more strings containing key/value pairs.
Data files keys/data files	Specifies one or more strings containing name/value pairs.
Start time	Stores the expected start time of the request.
Interval	Stores the number of seconds until the dispatcher request is translated again.
End time	Stores the end time of the request.

Dispatcher request states

The following information describes the various Dispatcher states associated with the translation process.

State	Description
INITIAL	Indicates the initial state of a newly created **DispatcherRequest** object.
PREPARING	Indicates that the data is being extracted from Teamcenter and that a Dispatcher Server task is being prepared.
SUPERSEDING	Supports the situation in which an original request contains a set of objects that require more than one translation—for example, if the set

State	Description
	contains both CATIA V4 and CATIA V5 parts. This state indicates that successor dispatcher requests are being created.
SCHEDULING	Indicates that the task is being added to the Dispatcher Server scheduling queue.
TRANSLATING	Indicates the task is being processed by the Dispatcher Server.
LOADING	Indicates the task results are being loaded into Teamcenter.
COMPLETE	Indicates the Dispatcher task has successfully completed.
TERMINAL	Indicates the Dispatcher tasks has failed.
DUPLICATE	Indicates the primary objects (datasets) specified in the **DispatcherRequest** are also specified by one or more **DispatcherRequest** objects currently being translated.
DELETE	Indicates the dispatcher object has been marked for deletion.
CANCELLED	Indicates the dispatcher request has been canceled.
SUPERSEDED	Indicates a successful end state in which the original dispatcher request is replaced by one or more newer requests.
NO_TRANS	Indicates a successful end state in which the original dispatcher request contained objects that did not require translation.

Each dispatcher request object has an *initial* state value, one or more *working* state values, and one or more *end* state values. Each dispatcher service processes **DispatcherRequest** objects whose current state matches the *initial* state of the dispatcher service. When processing by the dispatcher service completes, it sets the state of the dispatcher request to an *end* state value.

Dispatcher request states	Value
Initial	**INITIAL**
In Progress	**PREPARING**, **SCHEDULED**, **TRANSLATING**, **LOADING**, **SUPERSEDING**
Final	**COMPLETE**, **DUPLICATE**, **DELETE**, **CANCELLED**, **SUPERSEDED**, **NO_TRANS**, **TERMINAL**

Translation process

1. Based on actions such as checkin and workflow, one or more **DispatcherRequest** objects are created. (In Teamcenter, the **DispatcherRequest** object is represented by the **DispatcherRequest** class). The dispatcher requests are added to the **DispatcherRequest** database table with a state value of **INITIAL**.

2. Each dispatcher client queries the **DispatcherRequest** database table at the configuration-defined interval. The dispatcher client queries for **DispatcherRequest** objects with a state value of **INITIAL**.

3. The dispatcher client changes the **DispatcherRequest** state value to **PREPARING** to prevent other dispatcher client instances from operating on the same **DispatcherRequest** object.

4. The dispatcher request is validated to check whether it is a duplicate of another *in-process* request. If it is, the **DispatcherRequest** state value is set to **DUPLICATE**.

5. The translator-specific task preparation logic is activated to prepare data for translation input and create a task XML file to facilitate submission of the translation task to the scheduler. The input and XML files are collected in a task staging location. After task preparation is complete, the dispatcher client submits the task to the scheduler and sets the **DispatcherRequest** object state value to **SCHEDULED**. If an error occurs during preparation, the **DispatcherRequest** state value is set to **TERMINAL**.

6. When translation begins, the Dispatcher Server notifies the scheduler and the **DispatcherRequest** state is set to **TRANSLATING**.

7. When translation successfully completes, the Dispatcher Server again notifies the dispatcher client and the **DispatcherRequest** state value is set to **LOADING**. If an error occurs or the translation task fails, the Dispatcher Server notifies the scheduler and sets the **DispatcherRequest** state value to **TERMINAL**.

8. After the result files are loaded to the task staging location, the result files are mapped to Teamcenter application data. The dispatcher client then activates the appropriate translator integration logic to store the translation results in the Teamcenter database containing the associated Teamcenter application data model. When the results are loaded, the dispatcher client sets the **DispatcherRequest** state value to **COMPLETE**. If an error occurs during loading, the **DispatcherRequest** state value is set to **TERMINAL**.
   If the translation fails, the request is updated with a log file in the loader. This is based on configuration settings.

2. Installing Dispatcher

Installing Dispatcher

Install and configure Dispatcher using Teamcenter Environment Manager (TEM).

> Note:
>
> The following procedures describe the automated installation of Teamcenter Dispatcher components using TEM. If your configuration requires, you can also **manually install Dispatcher Server components**.

Install all Dispatcher components

> Note:
>
> The following instructions are for installing Dispatcher with a new installation of Teamcenter.

1. Start Teamcenter Environment Manager (TEM).
   If you are installing Dispatcher to an already existing Teamcenter server, launch Teamcenter Environment Manager from your Teamcenter environment.
   If this is a first-time installation of a Teamcenter server and you want to include Dispatcher, run the **TEM.bat** (Windows) or **TEM.sh** (UNIX) file from your installation source, such as a DVD.

2. In the **Solutions** panel, select **Dispatcher (Dispatcher Server)**.

3. Perform the following steps in the **Select Features** panel:

   a. Under **Enterprise Knowledge Foundation**, select the following:

   - **Dispatcher Client for Rich Client**
     Installs the dispatcher client on the rich client.
     This feature requires either Teamcenter two-tier rich client or Teamcenter four-tier rich client.

   - **Dispatcher Server**
     Installs Dispatcher Server components: scheduler, module, and Admin Client.

   - **Dispatcher Client**
     Installs the dispatcher client.
     This feature requires Teamcenter foundation and a two-tier version of Teamcenter.

4. Enter information as needed in subsequent panels.

5. In the **Dispatcher Components** panel, do the following:

   a. In the **Dispatcher Root Directory** box, type or select the Dispatcher root directory.
   Keep the Dispatcher root directory as close to the Teamcenter root directory as possible.
   This directory is referred to as *DISP_ROOT*.

   b. Select the **Install Scheduler** check box to install the scheduler.

   c. Select the **Install Module** check box to install the module.
   If you select the **Install Module** check box, the **Staging Directory** box is activated.
   In the **Staging Directory** box, you can choose the default staging directory or type a new one.

   > Note:
   >
   > If you are installing the module and the scheduler on the same machine, the **Scheduler Host** and the **Scheduler Port** boxes are disabled.

   d. Select the **Install Admin Client** check box to install the Admin Client.

   e. Click **Next**.

6. In the **Dispatcher Settings** panel, do the following:

   a. Select the logging level in the **Enter Logging Level** box.

   b. The **Dispatcher Services Log Directory** is automatically populated with the location of the log directory.

   c. Select the **Install Documentation** check box to install Javadocs for the Dispatcher components.

   d. In the **Documentation Install Directory** box, type or browse to the location where you want the documentation installed.

   e. If you want to start Dispatcher services after installation, select the **Starting Dispatcher Services** check box.

      - To start Dispatcher services as a Windows service, click **Start Dispatcher Services as Windows Services**.

      - To start Dispatcher services at the console, click **Start Dispatcher Services at console**.

   f. Click **Next**.

7. In the **Select Translators** panel, select the translators you want to enable.
   You can get more information about the different translators from the **translator readme files**.

Click **Next**.

8. In the **Translator Settings** panel, provide configuration information for the translators you selected in the **Select Translators** panel.
Click **Next**.

9. In the **Dispatcher Client** panel, do the following:

   a. Choose the **Dispatcher Server Connection Type** option. This option allows you to choose how the dispatcher client communicates between Teamcenter and the Dispatcher components.

      - Select **RMI** for communicating using the RMI mode.

        Note:

        The RMI mode of communication is faster than the WebServer mode.

      - Select **WebServer** if you send translation requests over a firewall.

   b. In the **Dispatcher Server Hostname** box, type the name of the server where the translation server will be hosted.

   c. In the **Dispatcher Server Port** box, type the port to be used for the Dispatcher Server. The default port number is **2001** if the value in the **Dispatcher Server Connection Type** box is **RMI** and **8080** if the value in the **Dispatcher Server Connection Type** box is **WebServer**. (In RMI mode, the scheduler port is used. In WebServer mode, the Web server port is used.) Make sure that the port you choose is not used by any other process.

   d. In the **Staging Directory** box, type or browse to the location to be used as the staging directory for the dispatcher client.

   e. In the **Dispatcher Client Proxy User Name** box, type the name of the proxy user who will use dispatcher services.

   f. In the **Dispatcher Client Proxy Password** box, type the password for the proxy user.

   g. In the **Dispatcher Client Proxy Confirm Password** box, type the password for the proxy user.

   h. In the **Polling interval in seconds** box, type the time in seconds that a dispatcher client should wait before querying for dispatcher requests.

   i. In the **Do you want to store JT files in Source Volume?** box, select **Yes** if you want to store visualization files in the associated visualization dataset.
   If you select **No**, the visualization files are stored in the *Dispatcher Client Proxy User* default volume.

j. Click **Next**.

10. In the **Dispatcher Client** panel, do the following:

    a. Select the logging level in the **Enter Logging Level** list.

    b. The **Dispatcher Client Log Directory** is automatically populated with the location of the log directory.

    c. In the **Advanced Settings** section, do the following:

        A. In the **Do you want to Update Existing Visualization Data?** box, select **Yes** if you want to update existing visualization data to the latest version.

        B. In the **Deletion of successful translation in minutes** box, specify the time (in minutes) that the dispatcher client should wait before querying and deleting successful translation request objects.
        If the interval is set to zero, translation request cleanup will not be done.

        C. In the **Threshold time in minutes for successful translation deletion** box, specify the time (in minutes) that must pass after a successful translation request object is last modified before it can be deleted.

        D. In the **Deletion of unsuccessful translation in minutes** box, specify the time (in minutes) that the dispatcher client should wait before querying and deleting unsuccessful translation request objects.

        E. In the **Threshold time in minutes for unsuccessful translation deletion** box, specify the time (in minutes) that must pass after an unsuccessful translation request object is last modified before it can be deleted.

    d. Click **Next**.

11. In the **Confirm Selections** panel, click **Next**.
    The installation starts.

Install Dispatcher components to an existing Teamcenter installation or on different machines

Install Dispatcher components to an existing Teamcenter installation or on different machines

If you are installing Dispatcher components to different machines, decide what components are to be installed on what machines. This should be based on machine resources and throughput requirements for the installation.

Install Dispatcher while creating a new configuration

1. Start Teamcenter Environment Manager.

2. In the **Maintenance** panel, choose the **Configuration Manager** option and click **Next**.
   Teamcenter Environment Manager displays the **Configuration Maintenance** panel.

3. In the **Configuration Maintenance** panel, choose the **Add new configuration** option and click **Next**.
   Teamcenter Environment Manager displays the **New Configuration** panel.

4. Enter a description of and unique ID for the configuration you are creating and click **Next**.
   Teamcenter Environment Manager displays the **Solutions** panel.
   Ensure that you select the **Dispatcher (Dispatcher Server)** option.

5. Select the components to include in the configuration and click **Next**.
   Teamcenter Environment Manager displays the **Select Features** panel.

6. In the **Select Features** panel, select the Dispatcher components.
   The Dispatcher components are available under **Extensions→Enterprise Knowledge Foundation.**
   From this point, the procedure is the same as for the first configuration in an installation.
   Teamcenter Environment Manager displays additional panels depending on the components you are including in the configuration.

Install Dispatcher while performing maintenance on an existing configuration

1. Start Teamcenter Environment Manager.

2. In the **Maintenance** panel, choose the **Configuration Manager** option and click **Next**.
   Teamcenter Environment Manager displays the **Configuration Maintenance** panel.

3. In the **Configuration Maintenance** panel, choose the **Perform maintenance on an existing configuration** option and click **Next**.
   Teamcenter Environment Manager displays the **Configuration Selection** panel.

4. From the list of configurations, select the configuration you want to modify and click **Next**.
   Teamcenter Environment Manager displays the **Feature Maintenance** panel.

5. Select **Add/Remove Features** and click **Next**.
   Teamcenter Environment Manager displays the **Select Features** panel.

6. In the **Select Features** panel, select the Dispatcher components.
   The Dispatcher components are available under **Extensions→Enterprise Knowledge Foundation.**

From this point, the procedure is the same as that for the first configuration in an installation. Teamcenter Environment Manager displays additional panels depending on the components you are including in the configuration.

Verifying Dispatcher installation

To verify that the Dispatcher installation was successful, check the following:

- The scheduler, module, and dispatcher client are installed, and you should be able to successfully start these services.
- If you installed dispatcher client for the rich client, you see the **Translation** menu in My Teamcenter.
- You are able to perform a test translation with the **tozipfile** translator.

> Note:
>
> In the *TC_ROOT* directory, you see a directory called **Dispatcher**. You should not use the contents of this directory for Dispatcher customizations or configurations.
>
> All Dispatcher components are installed in a separate directory (*DISP_ROOT*). This directory is defined during Dispatcher installation. You should use this directory for Dispatcher customizations or configurations.

3. Configuring Dispatcher

Configuring Dispatcher

After installing Dispatcher, you must create an access rule for the dispatcher client. You must also start dispatcher services and configure translators if you did not configure translators during installation.

See the following topics for a list of configurations to be performed.

Create a dispatcher client access rule

After installation, you must add a rule to the access rule tree permitting the translation service proxy user to update attributes of a **DispatcherRequest** object. If this access rule is not created correctly, the dispatcher client reports errors.

1. Log on to the Teamcenter rich client as a system administrator.

2. Open Access Manager.

3. In Access Manager, add the following rule to the rule tree under **Has Class (POM_application_object)→Working**:

   Condition =Has Class

   Value =DispatcherRequest

   ACL Name =DispatcherRequest

   ACE Type of Accessor =User

   ACE ID of Accessor =*DC-Proxy-User-ID*

   ACE Privilege =Write

   ACE Privilege Value =Grant

   ACE Privilege =Delete

   ACE Privilege Value =Grant

Note:

This rule applies only to the standard Access Manager rule tree. If custom rules are specified for the installation, your custom rules must be modified to provide the equivalent access of this rule. The *DC-Proxy-User* must be the same user that was specified during installation using the Teamcenter Environment Manager installation tool.

Manually configuring Dispatcher components

Manually configuring Dispatcher components

The Dispatcher components are configured when you install Dispatcher. However, if you want to tweak the configuration of the Dispatcher components after installation, you must manually edit the configuration files.

Manually configure dispatcher client

- Edit the **Service.properties** file and the **DispatcherClient.config** file located in the *DISP_ROOT* **\DispatcherClient\conf** directory.

Run dispatcher client as a Windows service

- Run the **runDispatcherClientWinService.bat** file from the *DISP_ROOT***DispatcherClient\bin** directory.
- From the Windows Services console, start the dispatcher client service.

Editing Dispatcher configuration files

- Scheduler
  Edit the **transcheduler.properties** file located in the *DISP_ROOT***Scheduler\conf** directory.
- Module
  Edit the **transmodule.properties** file located in the *DISP_ROOT***Module\conf** directory.
- Dispatcher Client
  Edit the **Service.properties** file and the **DispatcherClient.config** file located in the *DISP_ROOT* **\DispatcherClient\conf** directory.

Configure logging for dispatcher requests

To attach log files to dispatcher requests, update the following value in the **Service.properties** file.

The **Service.properties** file is located in the *DISP_ROOT*/**DispatherClient**/**Conf** directory.

- To attach log files to the dispatcher request when translation is successful, set **Translator**.*Provider name*.*Translator name*.**LogsForComplete** to **true**

  Example:

  Translator.SIEMENS.tozipfile.LogsForComplete=true

  Note:

  By default, this attribute is set to false. If you do not want to attach log files to the dispatcher request (for successful translation), remove the entry from the **Service.properties** file.

- In case of translation failure, the log is automatically attached to the dispatcher request.

Configure Dispatcher to work with Security Services

1. Install the dispatcher client using TEM. This ensures that Teamcenter creates a *Dispatcher Client Proxy User*.

2. Create the same *Dispatcher Client Proxy User* in LDAP.

3. Ensure that the values of the following properties in the **Service.properties** file match LDAP values:

   - **Service.Tc.Group=dba**

   - **Service.Tc.User**=*Dispatcher Client Proxy User*

4. Create an encrypted password file by running **encryptPass.bat/sh** *LDAP password for Dispatcher Client Proxy User*.

5. Start dispatcher client.

Set preferences for repeating tasks

- Set the **ETS.Repeating_UI.<ProviderName>.<ServiceName>** preference to **TRUE** to display the repeating tasks functionality.

  Note:

  Repeating task functionality is an expensive option. This functionality increases the load on the Dispatcher Server.

Start Dispatcher services

During installation of the Dispatcher components, you specified the Dispatcher root directory. This directory is referred to as *DISP_ROOT*.

To fully process a Dispatcher request, all of the following services must be running in this order: scheduler, module, and dispatcher client.

During installation of the Dispatcher feature, you have the option of installing the Dispatcher services either as a service or as a console process.

Start scheduler

- Run the following command:
  Windows systems:

  DISP_ROOT`/Scheduler/bin/runscheduler.bat`

  UNIX systems:

  DISP_ROOT`/Scheduler/bin/runscheduler.sh`

Start module

- Run the following command:
  Windows systems:

  DISP_ROOT`/Module/bin/runmodule.bat`

  UNIX systems:

  DISP_ROOT`/Module/bin/runmodule.sh`

Start dispatcher client

- Ensure that the scheduler and module services are running before starting the dispatcher client.
  Run the following command:
  Windows systems:

  DISP_ROOT`/DispatcherClient/bin/runDispatcherClient.bat`

  UNIX systems:

  DISP_ROOT`/DispatcherClient/bin/runDispatcherClient.sh`

Run dispatcher client as a Windows service

On Windows systems, you can optionally configure the dispatcher client as a Windows service.

> Note:
>
> If you run the dispatcher client as a service, Siemens PLM Software recommends running the scheduler and modules also as a service.

1. Set TC_DATA and TC_ROOT in the **System variables** section of the **Windows Environment Variables** dialog box.
   Examples:

   ```
   TC_DATA=C:\Progra~1\Siemens\tcdata

   TC_ROOT=C:\Progra~1\Siemens\Teamcenter8
   ```

   > Note:
   >
   > When you install Teamcenter, TEM automatically sets the FMS_HOME system variable.
   >
   > The Dispatcher Client service fails to start as a Windows service if you do not set the TC_DATA and TC_ROOT system variables.

2. Run the **runDispatcherClientWinService.bat** file from the **DispatcherClient\bin** directory.

3. From the Windows Services console, right-click **DispatcherClient***version* service and choose **Properties**.

4. In the **Log On** pane, choose the **This account** option to assign a logon account for the Dispatcher Client service.

   > Note:
   >
   > You must provide administrator privileges for the Dispatcher Client service.

5. Start the Dispatcher Scheduler.

6. Start the **DispatcherClient***version* service.

> Note:
>
> Start the scheduler and modules before you start the dispatcher client to avoid connection delays and translation failures.

Update Dispatcher components to work with UTF-8 configuration

When Teamcenter is set up to work in UTF-8, you can configure Dispatcher processes to support UTF-8. To do this, update the startup scripts for all Dispatcher components (Dispatcher Client, Scheduler, and Module) by adding the following JAVA VM option:

```
-Dfile.encoding=UTF-8
```

> Note:
>
> If your translator uses JAVA, you must update its startup script.

Setting up and configuring translators

Before you can start translating CAD files, you must set up and configure translators.

4. Using Dispatcher

Using Dispatcher

This topic describes how to perform translations on files and datasets and how to use the Dispatcher request administration console

Create a ZIP file for test purposes

Siemens PLM Software recommends that you do a test **tozipfile** translation to make sure Dispatcher components are working correctly.

> Note:
>
> Make sure that you have created a dispatcher client access rule and started the Dispatcher services.

1. In My Teamcenter, select an item revision.
2. Choose **File→New→Dataset** and in the **New Dataset** dialog box, select **MSWord** as the type for the new dataset.
3. In the **New Dataset** dialog box, click **Import** to import your Microsoft Word file.
4. In the **Import File** dialog box, select the Microsoft Word file you want to import into My Teamcenter and click **Import**.
5. Click **OK** to close the **New Dataset** dialog box.
   Your file is imported as the new dataset type.
6. Select the imported dataset and choose **Translation→Translate** to translate your Microsoft Word to ZIP format.
7. In the **Translation Selection** dialog box, choose the **tozipfile** service.
8. Click **Finish** to start the translation process.
9. (Optional) Choose **Translation→ Administrator Console –All** to see the progress of the translation.
10. After the translation is complete, the ZIP file appears in the item revision.

Creating a translation request

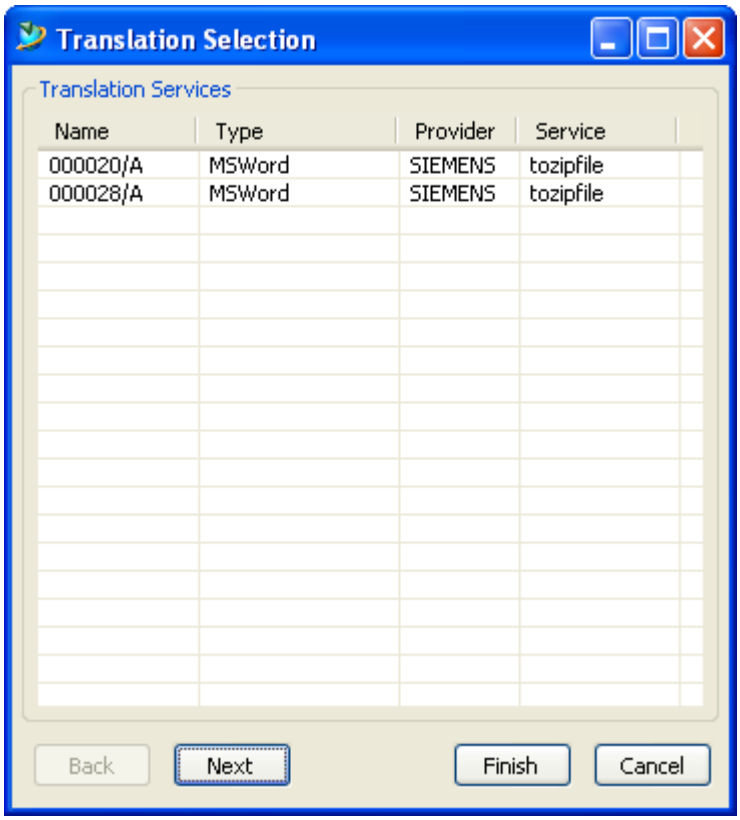
Creating a translation request

Dispatcher provides two ways to create a translation request:

- From the **Translation→Translate** menu command in My Teamcenter.
- Using the dispatcher_create_rqst utility.

Create translation requests in My Teamcenter

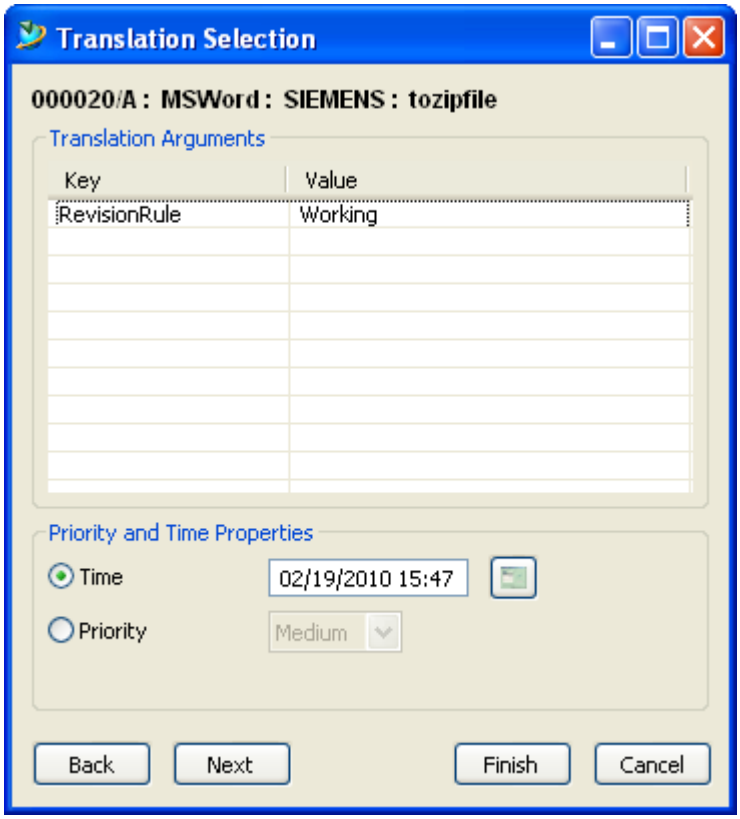
1. In the navigation pane, select one or multiple datasets, item revisions, or structure context objects for translations.
2. Choose **Translation→Translate**.
   The **Translation Selection** dialog box shows the selected objects for translation.
3. In the **Translation Selection** dialog box, choose appropriate values from the **Provider** and **Service** lists.



4. Click **Finish** to start the translation of all the objects.

The default translator arguments are used for the translation.

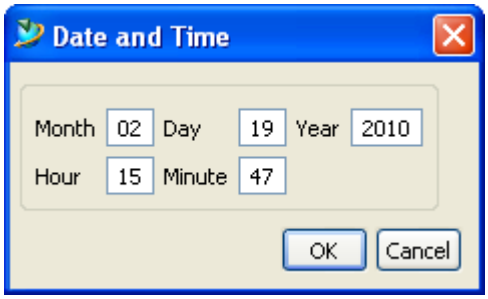
5. If you want to specify translator arguments and other properties, click **Next**.
Teamcenter shows the **Translation Selection** dialog box for the service.



6. In the **Translation Arguments** section, you add, modify, or delete **Key** and **Value** arguments.

7. In the **Priority and Time Properties** section, you can set the following options:

   a. **Time**
   Choose the time for the translation to start.
   Click the **Admin Time and Date properties** button to display the **Date and Time** dialog box.



In the **Date and Time** dialog box, type the translation start time and click **OK**.

b. **Priority**
Choose the priority for the translation task.

c. **Repeating**
Choose this option if you want to repeat the translation.

> Note:
>
> The **Repeating** option does not appear by default. You must set the **ETS.Repeating_UI.<ProviderName>.<ServiceName>** preference to **TRUE** to display the repeating tasks functionality.

> Note:
>
> To avoid unpredictable behavior, the (time) interval in repeating tasks must be greater than the translation time.

8. Click **Finish** to start the translation.
If there are other objects for translation, they are translated with the default values.

9. If you want to specify translator arguments and other properties for the remaining objects, click **Next**.

Create translation requests from the command line

You can create a translation request from the command line by using the **dispatcher_create_rqst** utility. This utility is located in the *TC_ROOT***/bin** directory.

You can get the usage details of this utility by typing the following on the command line:

```
dispatcher_create_rqst –h
```

Translate CAD files

1. In My Teamcenter, select an item revision.

2. In the item revision, select a CAD dataset and choose **Translation→Translate**.

3. In the **Translation Selection** dialog box, select the appropriate values for the **Provider** and **Service** lists.

4. Select the translation time, priority, and translation repeating schedule options from the **Date and Time Properties** section.

5. Click **OK** to start the translation.

6. (Optional) Choose **Translation→Administrator Console –All** to see the progress of the translation.

7. After the translation is complete, the translated CAD file appears in the item revision.

If your site has the Teamcenter lifecycle visualization embedded viewer installed, you can view the translation result in the **Viewer** data pane in My Teamcenter or in the **Viewer** tab in Structure Manager.

Using the Dispatcher request administration console

Using the Dispatcher request administration console

You can monitor and administer translation requests using the Dispatcher request administration console. To access the Dispatcher request administration console, in My Teamcenter choose **Translation→Administrator Console - All**.

You can perform the following actions:

- View all dispatcher requests.
- Resubmit a dispatcher request for processing.
- Delete a dispatcher request.
- View dispatcher properties.
- View dispatcher logs.

Dispatcher request administration console interface

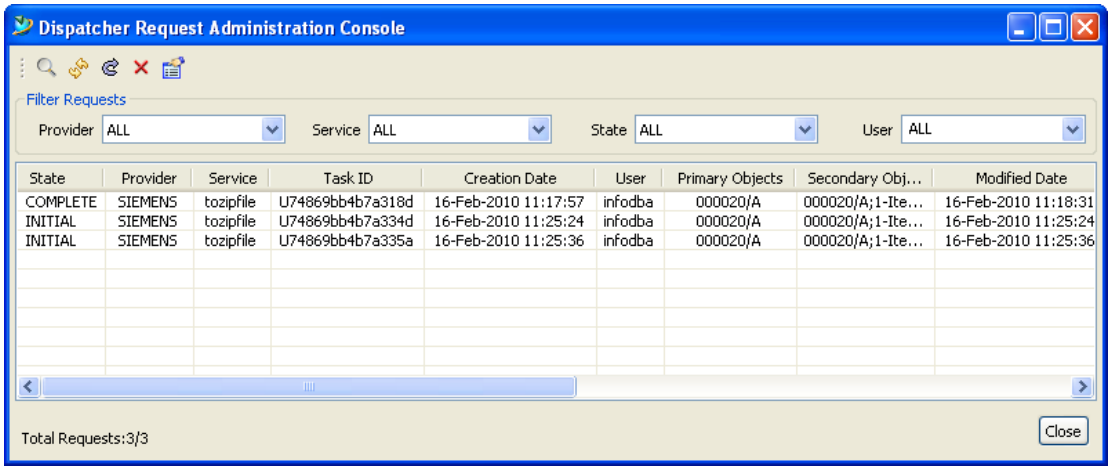


Figure 4-1. Dispatcher request administration console interface

Dispatcher request administration console buttons

Buttons	Description
	Refreshes all dispatcher requests.
	Refreshes the selected dispatcher request.
	Resubmits a dispatcher request for processing.
	Deletes the selected dispatcher request.
	Launches the properties dialog box for the selected dispatcher request.

Launch the Dispatcher request administration console

- In My Teamcenter, choose **Translation→Administrator Console - All**.

Refresh dispatcher requests

- In the **Dispatcher request administration console**, click one of the following:
  - **Refresh All Requests (SHIFT+F5)** button to refresh all dispatcher requests.
  - **Refresh Request (F5)** button to refresh the selected dispatcher request.

Resubmit a dispatcher request for processing

1. Select a dispatcher request.
2. Click the **Resubmit Request for Processing** button .

> Note:
>
> You can resubmit dispatcher requests only in final states.

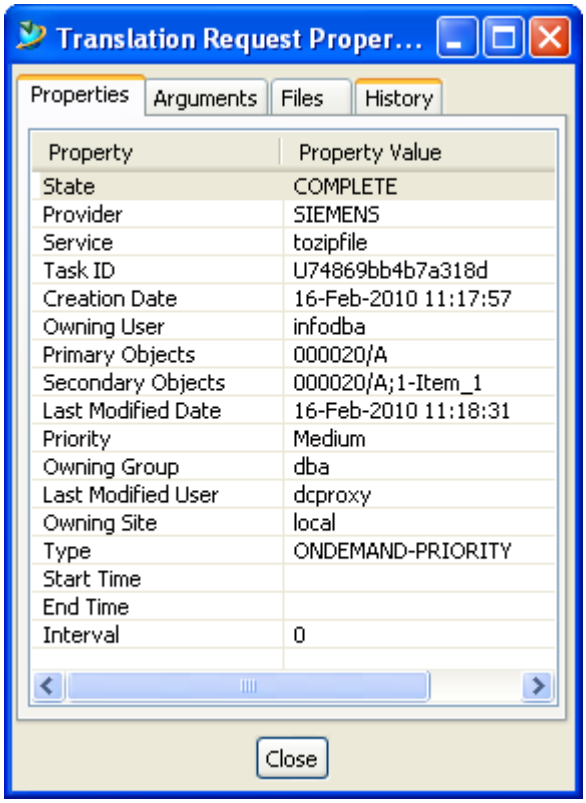
Delete dispatcher request

1. Select a dispatcher request.
2. Click the **Delete Request** button .
3. To delete dispatcher requests which are in-progress states , press the Shift key and click the **Delete Request** button .

> Note:
>
> The translation fails when you delete dispatcher requests which are in in-progress states.

View dispatcher properties

1. Choose a translation request and click the **Request Properties** button .
2. The **Translation Request Properties** dialog box appears.



a. Click the **Properties** tab to see the properties of the selected dispatcher.

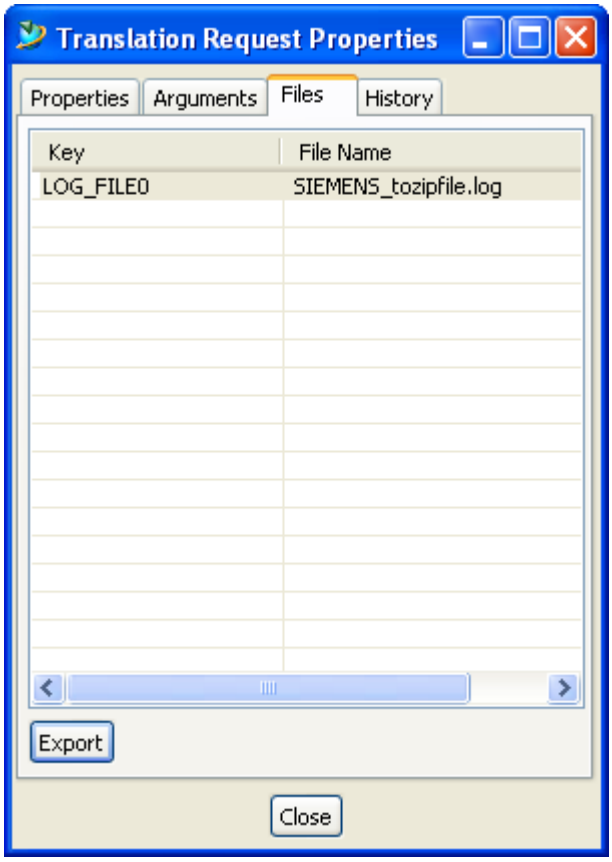
b. Click the **Arguments** tab to see the dispatcher arguments.

c. Click the **Files** tab to see the dispatcher log. You can download the dispatcher log to view it.

d. Click **History** to see a log of the different dispatcher states.

View logs attached to dispatcher requests

1. Choose a dispatcher request and click the **Request Properties** button .
2. In the **Translation Request Properties** dialog box, click the **Files** tab.
3. In the **Files** pane, select the log and click **Export** to view the log file locally.



Filter dispatcher requests

1. From the **Filter Requests** section, select the filters from the available lists.

2. Filter dispatcher requests by selecting options provided in the following lists:

   - **Provider**
   - **Service**
   - **State**
   - **User**

Dispatcher request administration console performance tuning

When you perform the **Refresh All** operation in the Dispatcher request administration console, if there are more than 5000 requests to retrieve, the **SOA** query has to make multiple server calls. This degrades performance.

To avoid this, set the **TC_SOA_MAX_PROPERTIES** environment variable from 100,000 to a higher number. For every additional 5000 requests the environment variable needs to be increased by 100,000.

Using logs for error handling

When you install Dispatcher using the TEM, you can specify a single root location for all log files. Logs are written in the following format for all the Dispatcher components.

- *Log-volume-location/category*/**process**
- *Log-volume-location/category*/**task**

Log-volume-location is the central repository of log files.

category refers to Dispatcher components.

process is the directory for all process logs. Process logs are the main log files for Dispatcher components.

task is the directory for all task logs. Task logs are log files specific to tasks such as submitting files for translation or generating translated files.

You can also view logs of a particular translation in the Dispatcher request administration console.

Stop, pause Dispatcher services

If you have installed Dispatcher services as a service, you can stop them by using the **stop** command.

Stop dispatcher client

- Run the following command:
  Windows systems:

  DISP_ROOT`/DispatcherClient/bin/runDispatcherClient.bat –stop`

  UNIX systems:

  DISP_ROOT`/DispatcherClient/bin/runDispatcherClient.sh –stop`

Stop module

- Run the following command:
  Windows systems:

DISP_ROOT/Module/bin/runmodule.bat –stop

UNIX systems:

DISP_ROOT/Module/bin/runmodule.sh –stop

Stop scheduler

- Run the following command:
  Windows systems:

  DISP_ROOT/Scheduler/bin/runscheduler.bat –stop

  UNIX systems:

  DISP_ROOT/Scheduler/bin/runscheduler.sh –stop

Pause dispatcher client

- The pause command stops processing requests that are in the **INITIAL** state. This means the dispatcher client does not accept any new task. Tasks which are *In Progress* states continue to be processed.
  To pause the dispatcher client, run the following command:
  Windows systems:

  DISP_ROOT/DispatcherClient/bin/runDispatcherClient.bat –pause true

  UNIX systems:

  DISP_ROOT/DispatcherClient/bin/runDispatcherClient.sh –pause true

- To start the dispatcher client after pause, run the following command:
  Windows systems:

  DISP_ROOT/DispatcherClient/bin/runDispatcherClient.bat –pause false

  UNIX systems:

  DISP_ROOT/DispatcherClient/bin/runDispatcherClient.sh –pause false

View Dispatcher service run-time information

You can view the run-time status of Dispatcher services using the ping command.

> Note:
>
> The ping command is performance intensive; use it only when required.

View run-time status of scheduler

- Run the following command:
  Windows systems:

  DISP_ROOT/Scheduler/bin/runscheduler.bat –ping

  UNIX systems:

  DISP_ROOT/Scheduler/bin/runscheduler.sh –ping

View run-time status of module

- Run the following command:
  Windows systems:

  DISP_ROOT/Module/bin/runmodule.bat –ping

  UNIX systems:

  DISP_ROOT/Module/bin/runmodule.sh –ping

View run-time status of dispatcher client

- Run the following command:
  Windows systems:

  DISP_ROOT/DispatcherClient/bin/runDispatcherClient.bat –ping

  UNIX systems:

  DISP_ROOT/DispatcherClient/bin/runDispatcherClient.sh –ping

View history of tasks performed by dispatcher client

- The history command parses the history logs and shows the detailed summary of tasks performed per day.
  Run the following command:
  Windows systems:

  DISP_ROOT/DispatcherClient/bin/runDispatcherClient.bat –history

UNIX systems:

```
DISP_ROOT/DispatcherClient/bin/runDispatcherClient.sh –history
```

5. Customizing Dispatcher

Customizing Dispatcher

This topic describes customizing the dispatcher client.

The dispatcher client provides a generic framework to support data translation including the triggering of translations, extraction of data from Teamcenter for translator input, translation of the data, storage of the translation results in Teamcenter, and management of the dispatcher request.

The dispatcher client provides limited support for some CAD manager data models and their respective translators. Within the dispatcher client, data model-specific logic is isolated into two areas: preparation of a translation task during extraction and storage of the translation task results during loading. CAD managers must provide full data model support as part of their dispatcher client integration to satisfy their specific data model and translator requirements.

The following dispatcher client customizations are supported:

- Adding new translators
  This consists of two parts:
  - Module changes
  - Dispatcher client changes
    - Dispatcher client behavior
      - ◊ Validating dispatcher requests
      - ◊ Coordinating external systems with dispatcher request processing
    - Translation triggers
      - ◊ Creating dispatcher requests in response to specific user events
    - Dispatcher client control
      - ◊ Modifying existing or creating new service filters.
    - Translator support
      - ◊ Modifying standard data preparation logic
      - ◊ Modifying the standard results store logic

◊ Adding new data preparation and storage logic

- Customizing dispatcher request creation
  - Create dispatcher requests in the Teamcenter rich client.
  - Create dispatcher requests using ITK APIs.
  - Create batch translations.
  - Create workflow translations.

Note:

The dispatcher client users Teamcenter Services from Teamcenter 8 and later.

You must change existing dispatcher client customizations (**Task Prep** and **DatabaseOperation** classes) written before Teamcenter 8 using rich client APIs . Use Teamcenter Services instead of rich client APIs.

Dispatcher client architecture

Dispatcher client architecture

The dispatcher client employs the ETL model (Extract-Transform-Load) for data translation. It provides a framework that supports extraction of data for translation, translation, and loading of translation results. The dispatcher client is implemented as a stand-alone SOA application.

- Extraction provides a collection of input data to be submitted as a translation task to the Dispatcher Server. The nature of the extracted data is dependent upon the input required by the desired translation.
- After extraction, the dispatcher client submits a translation task to the Dispatcher Server. The Dispatcher Server then processes the translation tasks.
- After the translation is complete, the dispatcher client loads the translated data in the appropriate Teamcenter target data model.
  The nature of the translation results is dependent on the output generated by the desired translation. Storage of the results is also dependent on the target data model.

The dispatcher client provides support for translator integrations and related data models with two Java classes: one for preparation of translator input (**TaskPrep** class) and one for storage of translator output (**DatabaseOperation** class). These two classes provide a structure for translator- and data model-specific logic implementation. The dispatcher client configuration properties define the mapping between translators and supporting classes.

These classes provide a structure for translator-specific and data model–specific logic implementation. The dispatcher client configuration properties define the mapping between translators and supporting classes.

DispatcherRequest object

DispatcherRequest object

The Dispatcher task execution process starts by the creation of a **DispatcherRequest** object. The **DispatcherRequest** object is represented by the **DispatcherRequest** class.

DispatcherRequest object attributes

Attribute	Description
Current state	Displays the current state of the dispatcher request.
Provider name	Displays the name of the provider of the translator.
Service name	Displays the name of the dispatcher service to be used.
Priority	Specifies a scheduling priority. Set it to low, medium, or high.
Task ID	Displays the task identifier assigned during the preparation process.
Creation date	Displays the date and time when the **DispatcherRequest** object was created.
Primary objects	Specifies one or more primary objects containing the data to be translated. When translating CAD data, primary objects are typically datasets.
Secondary objects	Specifies one or more supporting objects for the translation. Note: Secondary objects are optional and dependant on the requirements of the specific translation. When translating CAD data, secondary objects are typically item revisions.
Owning user	Displays the name of the user that created the request.
Owning group	Displays the name of the group that created the request.
History states	Displays various states that make up the history of the dispatcher request.
History dates	Displays timestamps associated with state values.
Type	Shows an application the purpose of the dispatcher request.
Mode	Represents a user-defined mode of operation.

Attribute	Description
Argument keys/argument data	Specifies one or more strings containing key/value pairs.
Data files keys/data files	Specifies one or more strings containing name/value pairs.
Start time	Stores the expected start time of the request.
Interval	Stores the number of seconds until the dispatcher request is translated again.
End time	Stores the end time of the request.

Dispatcher request states

The following information describes the various Dispatcher states associated with the translation process.

State	Description
INITIAL	Indicates the initial state of a newly created **DispatcherRequest** object.
PREPARING	Indicates that the data is being extracted from Teamcenter and that a Dispatcher Server task is being prepared.
SUPERSEDING	Supports the situation in which an original request contains a set of objects that require more than one translation—for example, if the set contains both CATIA V4 and CATIA V5 parts. This state indicates that successor dispatcher requests are being created.
SCHEDULING	Indicates that the task is being added to the Dispatcher Server scheduling queue.
TRANSLATING	Indicates the task is being processed by the Dispatcher Server.
LOADING	Indicates the task results are being loaded into Teamcenter.
COMPLETE	Indicates the Dispatcher task has successfully completed.
TERMINAL	Indicates the Dispatcher tasks has failed.
DUPLICATE	Indicates the primary objects (datasets) specified in the **DispatcherRequest** are also specified by one or more **DispatcherRequest** objects currently being translated.
DELETE	Indicates the dispatcher object has been marked for deletion.
CANCELLED	Indicates the dispatcher request has been canceled.

State	Description
SUPERSEDED	Indicates a successful end state in which the original dispatcher request is replaced by one or more newer requests.
NO_TRANS	Indicates a successful end state in which the original dispatcher request contained objects that did not require translation.

Each dispatcher request object has an *initial* state value, one or more *working* state values, and one or more *end* state values. Each dispatcher service processes **DispatcherRequest** objects whose current state matches the *initial* state of the dispatcher service. When processing by the dispatcher service completes, it sets the state of the dispatcher request to an *end* state value.

Dispatcher request states	Value
Initial	**INITIAL**
In Progress	**PREPARING**, **SCHEDULED**, **TRANSLATING**, **LOADING**, **SUPERSEDING**
Final	**COMPLETE**, **DUPLICATE**, **DELETE**, **CANCELLED**, **SUPERSEDED**, **NO_TRANS**, **TERMINAL**

Translation process

1. Based on actions such as checkin and workflow, one or more **DispatcherRequest** objects are created. (In Teamcenter, the **DispatcherRequest** object is represented by the **DispatcherRequest** class). The dispatcher requests are added to the **DispatcherRequest** database table with a state value of **INITIAL**.

2. Each dispatcher client queries the **DispatcherRequest** database table at the configuration-defined interval. The dispatcher client queries for **DispatcherRequest** objects with a state value of **INITIAL**.

3. The dispatcher client changes the **DispatcherRequest** state value to **PREPARING** to prevent other dispatcher client instances from operating on the same **DispatcherRequest** object.

4. The dispatcher request is validated to check whether it is a duplicate of another *in-process* request. If it is, the **DispatcherRequest** state value is set to **DUPLICATE**.

5. The translator-specific task preparation logic is activated to prepare data for translation input and create a task XML file to facilitate submission of the translation task to the scheduler. The input and XML files are collected in a task staging location. After task preparation is complete, the dispatcher client submits the task to the scheduler and sets the **DispatcherRequest** object state value to **SCHEDULED**. If an error occurs during preparation, the **DispatcherRequest** state value is set to **TERMINAL**.

6. When translation begins, the Dispatcher Server notifies the scheduler and the **DispatcherRequest** state is set to **TRANSLATING**.

7. When translation successfully completes, the Dispatcher Server again notifies the dispatcher client and the **DispatcherRequest** state value is set to **LOADING**. If an error occurs or the translation task fails, the Dispatcher Server notifies the scheduler and sets the **DispatcherRequest** state value to **TERMINAL**.

8. After the result files are loaded to the task staging location, the result files are mapped to Teamcenter application data. The dispatcher client then activates the appropriate translator integration logic to store the translation results in the Teamcenter database containing the associated Teamcenter application data model. When the results are loaded, the dispatcher client sets the **DispatcherRequest** state value to **COMPLETE**. If an error occurs during loading, the **DispatcherRequest** state value is set to **TERMINAL**.
   If the translation fails, the request is updated with a log file in the loader. This is based on configuration settings.

Adding a new translator to Dispatcher

Adding a new translator to Dispatcher

You can add your custom translators to Dispatcher. To add a new translator to Dispatcher, you must do the following:

- **Configure the module**.
- Configure the dispatcher client to integrate the new translator with Teamcenter rich client.

Dispatcher client changes

Dispatcher client changes

Dispatcher client changes include the following changes:

- Dispatcher client behavior
  - Validating dispatcher requests
  - Coordinating external systems with dispatcher request processing
- Translation triggers
  - Creating dispatcher requests in response to specific user events
- Dispatcher client control

Modifying existing or creating new service filters.

- Translator support
  - Modifying standard data preparation logic
  - Modifying the standard results store logic
  - Adding new data preparation and storage logic

Dispatcher client behavior

You can customize dispatcher client behavior using extensions. Extensions allow you to write a custom function or method for Teamcenter in C or C++ and attach the rules to predefined hook points in Teamcenter.

> Note:
>
> All user exits for Dispatcher must be replaced by extensions.

Dispatcher client control

Dispatcher client control

Validation

The dispatcher client does a duplicate check by default to make sure the same primary object is not sent for translation. You can define translator-specific duplicate check by setting the **Service.CheckForDuplicateRequests** property in the property file of the translator. If this property is not set for the translator, the default **Service.CheckForDuplicateRequests** property value will be used. Translator-specific duplicate check helps to override the default **Service.CheckForDuplicateRequests** property.

> Example:
>
> Translator.*provider name*.*translator name*.Duplicate=false

Filters

You can control translation services using dispatcher client filters. Dispatcher client filters provide a convenient way to control dispatcher request objects that are exposed to the dispatcher client for processing. If filters are specified for a dispatcher client, all dispatcher requests must pass this filter, or the dispatcher request is not processed by the dispatcher client.

Configuration of dispatcher client filter is specified in the **Service.properties** file, located in the *DISP_ROOT*/**DispatcherClient/conf** directory.

Dispatcher client filters are implemented as a subclasses of the **RequestFilter** class. Dispatcher supplies three request filters:

- **TranslatorFilter**
  Filters dispatcher requests that do not contain the specified providers or translators.

- **PriorityFilter**
  Filters dispatcher requests that have a priority lower than the specified priority.

- **OwningUserFilter**
  Filters dispatcher requests that do not match the specified owning user name.

Request filter class

Filters are implemented by subclasses of the **RequestFilter** class. Filter instances are created by the **FilterFactory**[1] class utilizing methods of the **RequestFilter** class. All translation services apply filtering to the results returned from the query for dispatcher requests.

To create a new filter, create a new subclass of the **RequestFilter** class providing an implementation for each of the methods specific to the new filter.

A sample **Filter.java** file is provided in the *DISP_ROOT*/**DispatcherClient/sample/integration/filters** directory.

RequestFilter contains the **properties** data member. The **properties** data member is an object that contains the configuration required and optional properties and values that are established when the dispatcher client starts.

The **RequestFilter** contains the following methods:

- **getRequiredPropertyNames**
  This method is used by the **FilterFactory** class to return an array of property names of required configuration properties. These properties are those that must be specified by the filter configuration. If no value is specified for these properties, the **FilterFactory** class logs an error in the Extractor service instance log file and no filter instance is created.

```
public abstract String[] getRequiredPropertyNames();
```

- **getOptionalPropertyNames**
  This method is used by the **FilterFactory** class to return an array of property names of optional configuration properties. These properties are optional.

```
public abstract String[] getOptionalPropertyNames();
```

1 **FilterFactory** is a class supplied by Dispatcher. It is a mechanism used by the service classes for creating instances of **RequestFilter** subclasses—for example, **PriorityFilter**.

- **setProperties**
  This method is used by the **FilterFactory** class to allow a specific **RequestFilter** subclass access to property values. The subclass implementation must execute the **super** method.

```
public void setProperties(Properties p) throws Exception
    {
        properties = p;
    }
```

- **filterRequest**
  This method is used to filter dispatcher request objects for processing by the dispatcher client. It is run by the dispatcher client to filter the results of the query for dispatcher request for processing.

```
public abstract boolean filterRequest( IMANComponent request )
throws Exception;
```

Develop custom dispatcher client filter

To customize a dispatcher client filter in a Microsoft Windows environment:

> Note:
>
> Adapt the procedure for UNIX as required.

1. Create the custom dispatcher client filter Java source file. Use a custom package designation.
   A sample **Filter.java** file is provided in the *DISP_ROOT***/DispatcherClient/sample/integration/filters** directory.

2. Ensure that the appropriate Java SDK is installed and Java path information is correctly set.

3. Compile the Java source file using the following command:

```
javac –classpath
  Add jars created in the classpath of runDispatcher.bat/sh file
  myFilter.java
```

4. Archive the new Java classes. These new classes, along with other customized dispatcher client classes, are combined into the CAD Manager JAR file or a single site-specific custom JAR file.

```
jar cf jar-file-name.jar myFile.class myFile2.class …
```

5. Copy the new JAR file to the *DISP_ROOT***/DispatcherClient/lib** directory.

6. Modify the *DISP_ROOT***/conf/Service.properties** file to specify the new filter and associated properties.
   Add the filter in the **FILTERS** section of the **Service.properties** file.

7. Run the dispatcher client and verify the operation of the filter.

Adding translator support

Overview of adding translator support

The dispatcher client provides the framework to support translators. Each translator has its own specific input and output requirements. Some translators require very little input and store their results directly in the Teamcenter database. Other translators require extensive input information and require result files to be stored appropriately.

Service configuration

Service configuration overview

The dispatcher client support for translators is specified as a set of translator provider and translator service class mappings. These mappings are described in the **Service.properties** file located in the *DISP_ROOT***/DispatcherClient/conf** directory.

Add translator to dispatcher client

1. Create a *Translator service name***Service.properties** file.
   For example: If you translator service name is **atob**, name the file **TSAToBService.properties**.

2. Add the following properties:

```
#====================================================================
# File description: This file contains the TS AToB service configuration
#
# Filename: TSAToBService.properties
#
#====================================================================
################################################################################
#
#     T R A N S L A T O R S
#
# See the Service.properties file for a detailed explanation of these properties.
#
# Provider Name  : SIEMENS
# Translator Name: AToB
Translator.SIEMENS.atob.Prepare=com.teamcenter.ets.translator.ugs.atob.TaskPrep
Translator.SIEMENS.atob.Load=com.teamcenter.ets.translator.ugs.atob.DatabaseOperatio
n
```

   Where:

   com.teamcenter.ets.translator.ugs.atob.TaskPrep is the class to extract the data from Teamcenter.

com.teamcenter.ets.translator.ugs.atob.DatabaseOperation is the class to load the translated data back to Teamcenter.

For more information, see the **Translators** section of the **Service.properties** file.

Set provider and service preferences

- If you have added a new provider, add the provider to the **ETS.PROVIDERS** preference.
- If you have added a new translator (service), add the translator to the **ETS.TRANSLATORS.<ProviderName>** preference.

Customizing data extraction and translation task preparation

Customizing data extraction and translation task preparation

Preparation of translator input is provided by the Teamcenter application or custom integration as an extension of the **TaskPrep** abstract task preparation class. The dispatcher client runs the **init**, **prepareTask**, and optionally, the **createSuccessorTranslationRequest** methods to prepare input data for dispatcher requests using the translator configuration-specified **Prepare** class for the provider name and translator name specified by the dispatcher request.

The **Prepare** class validates that all data necessary to translate the primary objects are available and collects the necessary files for input to the translator in a task staging location. Primary objects that are *data incomplete* are generally excluded from the translator input with appropriate logging messages. Objects that have translation results can also be excluded.

For more information, see the *DISP_ROOT***/DispatcherClient/docs/dc/javadoc** directory for **TaskPrep** class Java documentation.

Sample code for the **TaskPrep** class is provided in the *DISP_ROOT***/DispatcherClient/sample/integration/translators/proetojt** directory. A **DefaultTaskPrep** class is provided that handles the default behavior of the **TaskPrep** class. Customizations can be done by extending this class as shown in the sample code.

Sample code for the **DefaultTaskPrep** class is provided in the *DISP_ROOT***/DispatcherClient/sample/integration/translators** directory.

During the extract operation for a dispatcher request, the **TaskPrep** class methods are run by the dispatcher client in the following sequence:

Customizing data loading

Customizing data loading

Storage of translator results is provided by the Teamcenter application for custom integration as an extension of the **DatabaseOperation** abstract class. The dispatcher client runs the **DatabaseOperation** class methods to map result files using the configuration-specified **FileMap** class and store result data

for dispatcher requests using the configuration-specified **Load** class given for the dispatcher request provider name and translator name. The translator load integration class stores translation result files according to the Teamcenter application or custom integration data model.

For more information about the **DatabaseOperation** class, see the *DISP_ROOT*/**DispatcherClient/docs/ts/javadoc** directory.

During the load operation for a dispatcher request, the **DatabaseOperation** class methods are run by the Loader in the following sequence:

Add customizations during Dispatcher client startup

To call a custom code during the Dispatcher client startup, do the following:

1. Create a custom class that extends **com.teamcenter.ets.ServiceInit** and implement the **init** method.
   Example:

```
{
public void init()
  {
    //call custom code.
  }
}
```

2. Add the compiled class to the **DispatcherClient\lib** directory as a jar file.

3. Add the class name to **DispatcherClient\conf\Service.properties** as follows:

```
#Custom class to invoke the methods that need to be called during
DispatcherClient startup.
#This class needs to implement the "init" method defined in
com.teamcenter.ets.ServiceInitclass.ServiService.InitClass=
com.xyz.CustomInit
```

Configure logging

Dispatcher uses Log Manager for logging. Log Manager controls the location and other parameters during logging.

To create an SOA log file, use the method **ETSInit.createLogger("com.teamcenter.soa")**.

Integrate new translators in Teamcenter rich client

The following procedure explains how to integrate new translators in the Teamcenter rich client. It assumes that appropriate triggers for the translation already exist.

Note:

Adapt the procedure for UNIX as required.

1. Confirm that the desired translation is supported by a Dispatcher Server integrated translator.

2. Determine the following input requirements of the translator:

   - Data to be translated.
   - Datasets containing the data to be translated.
   - Named references are required to perform the translation.

3. Design the logic of the new **TaskPrep** subclass **prepareTask** method to satisfy translator input, and other Dispatcher requirements. Design the necessary helper classes. Sample **TaskPrep** classes and property files are provided in the *DISP_ROOT***/DispatcherClient/sample/integration/translator** directory.

4. Determine the preferences required by the new **TaskPrep** subclass and add them to the *Translator name***_env.xml** file. Keep in mind that some preferences may be required by the **TranslateMenu** classes and utilize these preferences before creating new ones.
   The *Translator-name***_env.xml** file should be created in the *DISP_ROOT***/DispatcherClient/install** directory.
   You must manually load the preferences into Teamcenter.

5. Determine the output of the translator.

6. Determine the Teamcenter data model under which the result files will be stored. Utilize the Teamcenter standard data model and supported extensions. Be sure the data model is supported by any downstream tools that are to consume translation results from the Teamcenter database.

7. Determine the **DatabaseOperation** method implementations that will be necessary to load the translator results in the Teamcenter database. This includes the **load** method at a minimum.

8. Design the logic of the required **DatabaseOperation** methods and any necessary helper classes. Sample **DatabaseOperation** classes and property files are provided in the *DISP_ROOT***/DispatcherClient/sample/integration/translator** directory.

9. Determine the preferences required by the new **DatabaseOperation** subclass and add them to the *Translator-name***_env.xml** file. Keep in mind that some preferences may be required by the **TranslateMenu** classes and utilize these preferences before creating new ones.

10. Extend the dispatcher client **DefaultTaskPrep** abstract class in a new **TaskPrep** class in a package specific to the translator.
    For example: *your path*.**translator.***provider name*.*translator name*

11. Implement the new **DatabaseOperation** subclass in the same package with the **TaskPrep** class.

12. Implement the associated helper classes.

13. Compile the Java source files.

    ```
    javac -classpath Classpath created by the
    runDispatcherClient.bat/sh file myFile.java
    ```

14. Create the **TS***Translator name***Service.properties** file to incorporate the new translator integration classes.
    The *provider* and *service* attribute values specified by the module translator configuration in the **translator.xml** file must be used in constructing the property names (keys).
    Edit the **Service.properties** file to import the new **TS***Translator name***Service.properties** file.

15. Archive the new Java classes and the **TS***Translator name***Service.properties** file. These new classes along with other customized dispatcher client classes are most simply combined into a single application or custom JAR file.
    Make sure the **TS***Translator name***Service.properties** file is in the root directory.

    ```
    jar cf jar file name.jar myFile.class
    myFile2.class TSTranslator nameService.properties
    ```

16. If the JAR file contains only translator integration classes, copy the new JAR file to the *DISP_ROOT*/**DispatcherClient/lib** directory.

17. Implement the new translator site preference XML file and use the **preferences_manager** executable to import the file.

    ```
    preferences_manager -u=infodba -p=infodba -g=dba -mode=import
     -scope=SITE -action=override -context=Teamcenter -file=your XML file
    ```

18. Modify the **ETS_trans_rqst_referenced_dataset_types** site preference to add the application or customized dataset type name of the datasets that may be specified as a primary objects in the dispatcher request for this translator. Addition of a dataset type to this preference allows cancellation of a dispatcher request and subsequent deletion of the primary objects while the translation is *in process*.

19. Add the **Translate** menu support for the new translation.

20. Start the dispatcher client.

21. Trigger an on-demand dispatcher request and verify that the new **TaskPrep** class run by the dispatcher client processes the dispatcher request correctly. Examine the contents of the Dispatcher Server file server task directory (or task staging location if configured for shared staging location). Verify that the XML task file is present and that its contents are correct. Verify that all the expected translator input files are present.

22. Verify that the translation task is successfully submitted to the Dispatcher Server scheduler.

23. When translation completes, verify the contents of the Dispatcher Server task directory (or task staging location) to see that all expected result files are present.

24. Verify that the new **DatabaseOperation** class runs by the dispatcher client processes the dispatcher request and correctly loads the result files. Confirm that the location and ownership of the stored results is correct.

25. Confirm that the results can be accessed in the Teamcenter database.

Creating dispatcher requests

Creating dispatcher requests

You can customize dispatcher request creation by:

- Adding translator preferences to the Teamcenter database and then using the **Translate** menu commands for translations.

- Using APIs for creating dispatcher requests. You can use these APIs for batch processing of dispatcher requests or creating a dispatcher request from a workflow.

Creating dispatcher requests in Teamcenter rich client

Dispatcher client provides a single dispatcher request creation menu that is driven from preferences defined within Teamcenter. This event is activated by selecting the **Translation→Translate** menu command in My Teamcenter through one of the specific translator menu selections.

To add the translator to the **Translate** menu command, you must add translation preferences to Teamcenter.

If this menu command does not allow the creation mechanism you need in the rich client, a method is exposed to create a dispatcher request on the **com.teamcenter.rac.ets** plug-in within the rich client. The package, class, and method to create the request are:

Package: **external**

Class: **DispatcherRequestFactory**

Method: **createDispatcherRequest**

Creating dispatcher requests using ITK APIs

Creating dispatcher requests using ITK APIs

Dispatcher client provides server APIs to support customized triggering of translations. The dispatcher client framework for creation of dispatcher requests allows customizations in the following areas:

- Batch translations
- Workflow

You can create dispatcher request objects using the **DISPATCHER_create_request** API.

These API support the creation of dispatcher request object in a server environment.

You must include the **dispatcher_itk.h** file to provide the prototype for these Dispatcher API functions, and you must modify link scripts to link against the **libdispatcher** shared object.

Create dispatcher requests using ITK APIs

1. Modify the sample **dispatcher_create_rqst_itk_main.c** source code or create a new ITK command line C source file.

2. Compile the source module using the sample script provided in the **TC_ROOT/sample** directory by entering the following command:

   ```
   compile -DIPLIB=none dispatcher_create_
   rqst_itk_main.c
   ```

3. Copy the sample ITK link script provided in the **TC_ROOT/sample** directory. Edit the copied ITK link script to add the Dispatcher library, **libdispatcher.lib**, to the **link** command.

4. Run the modified ITK link script by entering the following command:

   ```
   myLinkitk -o dispatcher_create_rqst_itk_main
   dispatcher_create_rqst_itk_main.obj
   ```

5. Test the new ITK executable.

Batch translations

Existing non-Dispatcher customer translation processes typically employ a daily process that runs during off hours to examine the contents of the customers Teamcenter database and create the necessary visualization data. You can integrate this process with Dispatcher using Dispatcher Integration Toolkit (ITK) APIs.

For performing batch translations, the following functionalities are provided:

- ITK functions for use in custom batch programs.
- Command line ITK executables (with sample source) for use by batch scripts.

Dispatcher provides the **DISPATCHER_create_request** server ITK API. This ITK API supports the creation of dispatcher request objects in a server environment.

Workflow translations

No workflow action handlers or triggers are currently provided by Dispatcher. Because workflow is a customer process, it is intended that on-site customizations of workflow action handlers use the Dispatcher Integration Toolkit (ITK) API to create dispatcher requests as they are needed.

Creating dispatcher requests using Teamcenter Services

Dispatcher provides a service to allow the creation of a dispatcher request using the service-oriented architecture within Teamcenter.

Translation menu integration

Translation menu integration

The **Translation** menu item appears on the menu bar, if you select the **Translation for Rich Client** option during installation. The following procedures describe how to create the new menu items in the **Translation** menu bar and to start a new dispatcher request using the new menu item.

Create new menu item

- Open **plugin.xml** in an Eclipse plugin project. In the **menuContribution** tag, add the following location;

```
<menuContribution
locationURI="menu:com.teamcenter.rac.ets.external.translate?
after=additions">
</menuContribution>
```

> Note:
>
> To place your new menu item after the **Translate...** menu item, use **after=additions** in the **menuContribution** tag.
>
> To place your new menu item before the **Translate...** menu item, use **after=bottom** in the **menuContribution** tag.

Create dispatcher request using the new menu item

- To create a new dispatcher request from the new menu item, use the **createDispatcherRequest** method as an exported function from the **DispatcherRequestFactory** class. This method allows you to create a dispatcher request easily from within your own plug-in by just referencing the main Dispatcher plug-in from Eclipse.

```
/**
     * Creates the DispatcherRequest.
     *
     * This function allows customers who wish to create a DispatcherRequest
     * through a menu or other plug-in within Teamcenter.
     *
     * [R] = required, [O] = optional
     *
     * @param [R] provider - string representing the name of the translator
     *                       provider
     * @param [R] service - string representing the name of the translator
     *                      to use
     * @param [R] priority - integer representing the priority used in
     *                       processing the request
     * @param [O] primaryObjects - array of components selected within
     *                             Teamcenter
     * @param [O] secondaryObjects - array of parent components for the
     *                               primary components
     * @param [O] startTime - time to start the request
     * @param [O] interval - number of times to repeat request
     * @param [O] endTime - time at which to stop repeating tasks
     * @param [O] type - string for user to define user specific type of request
     * @param [O] requestArgs - name/value arguments to pass to service
     *
     * @return true, if create DispatcherRequest
     *
     * @throws TCException the TC exception
     *
     * @published
     */
    boolean createDispatcherRequest(String provider, String service, int priority,
     TCComponent[] primaryObjects,
     TCComponent[] secondaryObjects,
     String startTime, int interval, String endTime,
     String type, Map<String, String> requestArgs) throws Exception
```

Add Dispatcher preferences

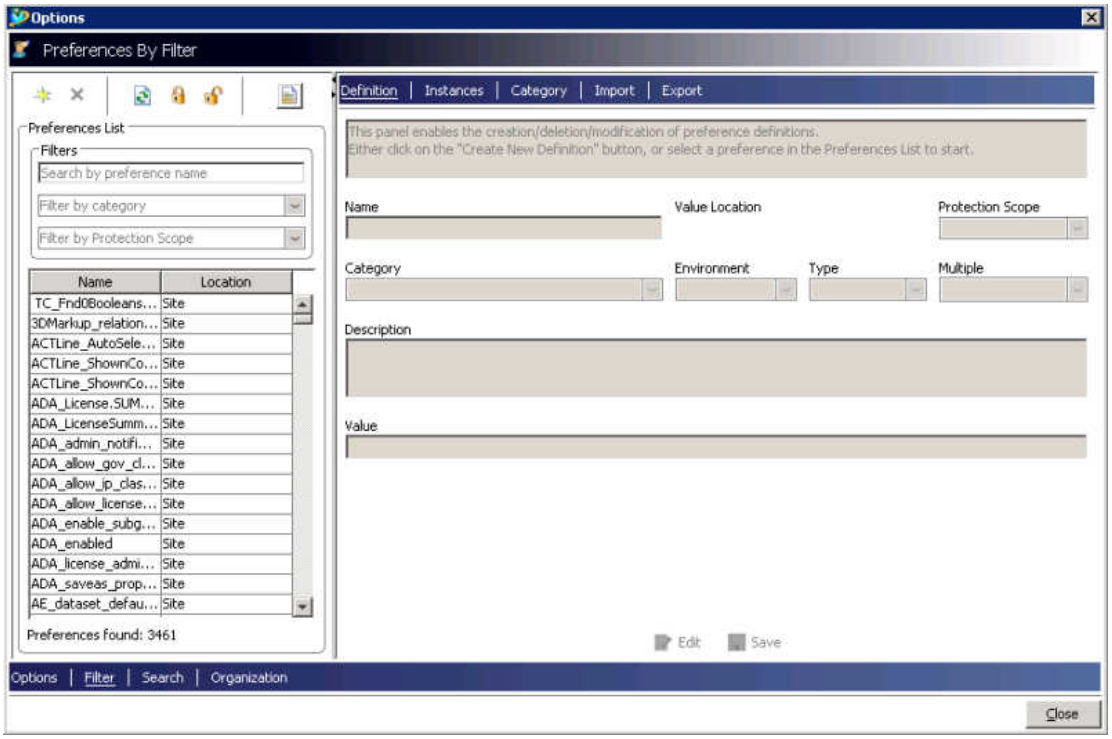
For translation options to appear in the **Translation Selection** dialog box, you must add Dispatcher preferences.

This dialog box allows you to select the translation provider and service for a selected data type. You can view the **Translation Selection** dialog box by choosing the **Translation→Translate** menu option in My Teamcenter menu.

> Note:
>
> The translator preferences provided by Dispatcher are imported in Teamcenter during installation. You only need to add preferences of new translators or providers that you want to add to Teamcenter.

1. In My Teamcenter, choose **Edit→Options**.

2. In the **Options** dialog box, select **Filter**. This option is located at the bottom left corner of the **Options** dialog box.
   The **Preferences By Filter** dialog box appears.



3. You can either modify the values of the existing preferences or add new preferences. To create new Dispatcher preferences, click the **Definition** tab and click **Create a new preference definition** in the **Preferences By Filter** dialog box.
   To modify the values of an existing preference, select the preference and change the values in the **Values** box.

   > Note:
   >
   > Access to preferences and preference hierarchy are specified by protection scope.

   You must set the following Dispatcher preferences:

Preference name	Description
ETS.PROVIDERS	Specifies the company that provides translators.
ETS.TRANSLATORS. *PROVIDER*	Specifies a list of translators for a specific provider.
ETS.DATASETTYPES. *PROVIDER.TRANSLATOR*	Specifies valid datasets for the selected translators. Note: Only the datasets specified in this preference are valid for translation.
ETS.PRIORITY. *PROVIDER.TRANSLATOR*	Specifies the priority to be assigned to a dispatcher request when the translator specified in this preference is used.
ETS.TRANSLATOR_ARGS. *PROVIDER.TRANSLATOR*	Specifies the list of translation arguments to be passed to the translator.

Note:

The order of values in the **Values** box determines the order in which they appear in the **Translation options** dialog box.

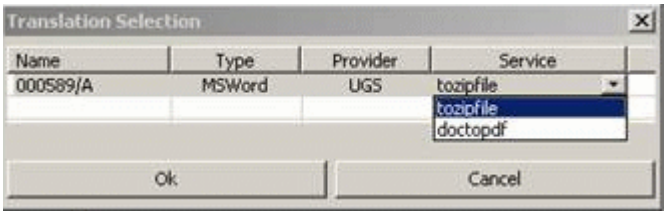
4. Restart the Teamcenter server and rich client.

5. Select a dataset for which you defined preferences, and choose **Translation→Translate**. The translators defined for the dataset appear.

Note:

If there is more than one translator definition, all the translators appear in a list.

The list with the translator values appears only if you select the cell that contains the translator service.



Customizing Translation Selection dialog box behavior

Customizing Translation Selection dialog box behavior

You can customize the **Translation Selection** dialog box behavior to do the following:

- Validate data.
- Dynamically add translation argument values to the dispatcher request.

You must perform the following steps to customize the **Translation Selection** dialog box.

- Create a custom plugin.
- Create a custom class.

Create a custom plugin

1. Create a new plugin based on the **com.teamcenter.rac.ets.translator.singleselection** plugin and extend the **requestCustomize** point.

2. You must provide the following information in the **plugin.xml** file.

   - **providerName**=*Name of the provider whose behavior needs to be customized*
   - **serviceName**=*Name of the service whose behavior needs to be customized*
   - **class**=*Name of the class that implements the customization*

   Your **plugin.xml** file looks like this:

```
<?xml version="1.0" encoding="UTF-8"?>
<?eclipse version="3.4"?>
<plugin>
  <extension point="com.teamcenter.rac.ets.translator.
singleselection.requestCustomize">
    <translator class=" com.foo.customization.abctoxyz.
AbcToXyzRequestCustomization"
      providerName="SIEMENS"
      serviceName="abctoxyz">
    </translator>
  </extension>
</plugin>
```

You can add multiple translator tags to provide customization classes for different services, and the same plugin can be used to provide customization classes for different services.

However, there can be only one customization class for a service and provider combination.

Create a custom class

1. Create a class **com.teamcenter.rac.dispatcher.translator.singleselection.customization.***translator service name.New class name***RequestCustomization**, which implements **com.teamcenter.rac.ets.translator.singleselection.customization.**

IRequestCustomizable.

2. This class must implement the following methods:

   - **validateRequestData**
   - **getValuesForKeys**

For more information about the extension point, refer to the **requestCustomize.exsd** schema located in the **com.teamcenter.rac.ets.translator.singleselection** plugin.

For an example on how to extend the **requestCustomize** extension point, see the **com.teamcenter.rac.dispatcher.customization** plugin.

Customizing the Dispatcher request administration console

Customizing the Dispatcher request administration console

Customizing the Dispatcher request administration console

You can customize the Dispatcher request administration console to:

- Filter requests in the Dispatcher request administration console.
  For example: Filter out other site requests, filter out requests owned by other users.

- Create a new sub-menu under the **Translation** menu.
  This menu launches the Dispatcher request administration console with predefined filtered results.
  For example: Create a sub-menu called *Terminal only* to show only terminal requests in the Dispatcher request administration console.

Filter requests in the Dispatcher request administration console

1. Create a new plugin based on the **com.teamcenter.rac.ets** plugin and extend the **DispatcherAdminConsoleFilter** point.

2. You must provide the following information in the **plugin.xml** file.
   class=com.teamcenter.rac.dispatcher.sampledispaccustomization.handlers.NonTerminalFilter
   Class is the actual class provided by new plugin which implements methods defined in the **com.teamcenter.rac.ets.customization.IDispatcherACCustomizable** interface.
   Your plugin.xml file looks like this:

```
<?xml version="1.0" encoding="UTF-8"?>
<?eclipse version="3.4"?>
<plugin>
  <extension
    id="adminconsolenonterminal"
    point="com.teamcenter.rac.ets.DispatcherAdminConsoleFilter">
    <filter
```

```
            class="com.teamcenter.rac.dispatcher.sampledispaccustomization.
                   handlers.NonTerminalFilter">
          </filter>
        </extension>
      </plugin>
```

There can be only one customization class for a service and provider combination. If there are multiple customizations, the Dispatcher request administration console ignores all the customizations and displays all the queried requests to user.

3. Create a class **com.teamcenter.rac.dispatcher.sampledispaccustomization.-handlers.NonTerminalFilter** which implements **com.teamcenter.rac.ets.customization.IDispatcherACCustomizable**.

4. This class must implement the method in **IDispatcherACCustomizable**.

For more information about the extension point, refer to the **com.teamcenter.rac.ets.DispatcherAdminConsoleFilter.exsd** schema in **com.teamcenter.rac.ets** plugin.

Create a new sub-menu under the Translation menu

1. Create a new plugin based on the **com.teamcenter.rac.ets** plugin.

2. In the new plugin, create a new menu and create a handler class to handle the menu click.

3. Create a class **com.teamcenter.rac.dispatcher.sampledispaccustomization.-handlers.NonTerminalFilter** which implements **com.teamcenter.rac.ets.customization.IDispatcherACCustomizable**.

4. This class must implement the method in **IDispatcherACCustomizable**.

5. In the handler class, implement **execute(ExecutionEvent)** method to call **AdminDialogService.OpenAdminDialog()** method and pass an instance of filter class created previously.

> Note:
>
> If you use this method, the Dispatcher request administration console will ignore filter class defined by extending the extension point. Dispatcher request administration console will only use the filter class passed in **OpenAdminDialog()** method.

A. Troubleshooting

Troubleshooting

If errors occur during translations, follow these guidelines:

- Ensure you are doing things correctly and the prerequisites are set up.
- Isolate the problem and then fix it.

Checklist to handle errors

- Ensure that the installation and configurations are properly done.
- Follow instructions in the documentation according to the order of dependencies.

  > Example:
  >
  > To run the **ideastojt** translator, ensure that the **ideastojt** translator is installed according to the I-deas installation guide.

- Check whether all the property files are configured correctly.
  When you are not sure of the property values, try to be as close as possible to the default property values.
- Ensure that all prerequisite software is installed and tested before installing the dispatcher client. Also ensure the correct versions of the prerequisite software are installed.
- Restart the dispatcher client and rich client after any property changes.
- Ensure that service access rules are added.
- Ensure that you have permissions to write to the staging directory.
- Ensure that you have file storage space to run translation services.

Isolate translation problems

1. To isolate the application that is causing the translation failure, run the translator in different contexts:

   a. Run the translator in a stand-alone mode from the command line. For information about running the translator from the command line, see the translator documentation.

It the translator fails, it is a translator issue. See the translator documentation for information about fixing it.
If the translator works, go to the next step.

b. Perform a translation within the context of Dispatcher Server using the administration client.
If the translation fails, it is a Dispatcher Server issue.
If the translation works, go to the next step.

c. Perform a translation within Teamcenter.
If the translation fails, it is a Teamcenter dispatcher client issue.

2. After the issue is identified as a Teamcenter dispatcher client issue, you can further isolate the problem by setting debug levels to higher values in the **log4j.xml** file.
More debug information is available in:

- Console in which the dispatcher client started.
- Service logs are available in the *Log-Directory***/Dispatcher/process/DispatcherClient.log** file.
- Task logs are available in the *Log-Directory***/Dispatcher/***Task-ID*/*Task-ID***_ts.log** directory.

3. Delete or back up old logs to ensure that you are not looking at wrong files for debug messages.

4. Create a new query for **DispatcherRequest** using Query Builder. This is useful to check whether the **DispatcherRequest** object is created in the Teamcenter database.

5. Shut down the dispatcher client. Use the on-demand dispatcher client to trigger a translation request.

a. Start the dispatcher client. Check the console, the **DispatcherClient.log** file, or task log files to see whether the dispatcher client was successful in extracting files from Teamcenter and uploading them to a staging location. Confirm that files are in a staging directory. If extraction does not occur for the translation request, query **DispatcherRequest** in the rich client to see whether a **DispatcherRequest** object is created.

b. After extraction, the task is submitted to the scheduler. Check whether the task was successfully submitted to the scheduler and that you get back a translation successful event from Dispatcher Server. The Dispatcher Server events should appear in the following order:

A. Submitted the task successfully

B. Started translation

C. Completed successfully

Confirm that result files (JT, CGM) are available on the file server staging directory. If translation fails, you get a translation error event. If translation fails, use input files in the file

server staging directory to translate independent of dispatcher client using Dispatcher Server administration client or the core stand-alone translator.
If translation fails, the input files have some problem.

c. Check whether the dispatcher client was successful in downloading result files and attaching result files back to Teamcenter. If the download is successful and the attachment failed, check whether file mapping failed (file mapping fails if result files do not map one on one with source files). This can occur if translators are not configured per dispatcher client requirements.

Setting debug levels

For all Dispatcher services, the **LogVolumeLocation** property defines the location of the log files. This property can be found in the following locations:

- Module
  DISP_ROOT/**Moduel/conf/transmodule.properties**

- Scheduler
  DISP_ROOT/**Scheduler/conf/transscheduler.properties**

- Dispatcher client
  DISP_ROOT/**DispatcherClient/conf/DispatcherClient.properties**

You can define the debug level for logs in the **Log4j.xml** file. This file is located in the *DISP_ROOT*/*Dispatchert-component*/**conf** directory.

To change the debug level, change the value of the **level value** property to the debug level you want.

Information about the supported debug levels is as follows.

```
<!-- Supported Levels of logging are -->

<!-- "OFF"      - The OFF Level is intended to turn off logging. -->

<!-- "FATAL"    - The FATAL level designates very severe error

     events that will presumably lead the application to abort. --

<!-- "ERROR"    - The ERROR level designates error events that might
still
 allow the application to continue running. -->

<!-- "WARN"     - The WARN level designates potentially harmful
situations. -->
```

```
<!-- "INFO"     - The INFO level designates informational messages that

 highlight the progress of the application at coarse-grained level. -->

<!-- "DEBUG"    - The DEBUG Level designates fine-grained informational
events
that are most useful to debug an application. -->

<!-- "TRACE"    - The TRACE Level designates finer-grained informational
events
than the DEBUG and tracks the start and end of method calls. -->

<!-- "${DBGHG}" - The ${DBGHG} Level designates very fine-grained
information

 than TRACE. Used to debug code which has lots of output. -->

<!-- "ALL"      - The ALL Level is intended to turn on all logging. -->

<logger name="Process">

<level value="INFO"/>

<appender-ref ref="ProcessAppender"/>

<appender-ref ref="ProcessConsoleAppender"/>

</logger>

<logger name="Task">

<level value="INFO"/>

<appender-ref ref="TaskAppender"/>

<appender-ref ref="TaskConsoleAppender"/

</logger>
```

Query translation request objects

To check whether a translation request object is created in the database, create a new query in the Query Builder application as follows:

1. In the **Query Builder** pane, type **DispatcherRequest** in the **Name** box.
   This specifies the name for the query.

2. Click the **Search Class** button  , and select **DispatcherRequest** as the search class.

3. In the **Attribute Selection** pane, double-click the **State** attribute to add the **State** attribute to the **Search Criteria** table.

4. In the **Search Criteria** pane, set the **Default Value** attribute to ***.

5. Click the **Create** button to create the query definition.

Unable to run Dispatcher components as Windows service due to missing DLL file

You cannot run Dispatcher components as Windows service if the **msvcr71.dll** file is missing from your Windows system directory. To add this file to the Windows system directory:

- Copy the **msvcr71.dll** file from the *Teamcenter-install-kit***install\install\jre\bin** directory to your Windows system directory.
  For example, copy **msvcr71.dll** to **C:\Windows\SysWOW64**.
  OR

- Add the *Teamcenter-install-kit***install\install\jre\bin** directory to the system path variable.

Optimizing performance of the dispatcher client

The dispatcher client performance may be impacted if you have a large number of dispatcher requests in the **INITIAL** state, for example more than 10000 dispatcher requests. If you observe performance issues, create a preference named **DISPATCHERCLIENT_MAX_TO_QUERY** with protection scope of **User** and a value of **500**. This preference limits the number of dispatcher requests that dispatcher client queries.

To create this preference, log in as the Dispatcher proxy user

You can adjust the value of this preference based on the Dispatcher performance.

Dispatcher request go to terminal state due to large number of files open

On a linux server, if the number of open dispatcher requests is greater than the number of files the server can handle, the extra dispatcher requests go to the terminal state.

To fix this, run the **ulimit** utility and update the **nofile** argument with the number of files or requests you want Dispatcher to handle at a time.

Example:

```
$ ulimit -n 65536
```

B. Service.properties file reference

The **Service.properties** file is located in the **DispatcherClient\conf** directory.

Property Name	Description	Valid values	Default values
Service.Tc.Host	The computer name where the dispatcher client tcserver processes will run.	String value corresponding to the computer name.	
Service.Tc.Marker	The Teamcenter database to log on to.	String value corresponding to the database server name.	
Service.Tc.User	The Teamcenter user that the dispatcher client logs on as. This user is also known as DC Proxy User.	String value corresponding to the Teamcenter user name.	
Service.Tc.Group	The Teamcenter group that the dispatcher client logs on as. This must be the dba group.	String value corresponding to the Teamcenter group name.	**dba**
Service.Tc.Port	Port number used for communication between dispatcher client and Teamcenter processes.	Any available port number.	
Service.Tc.ReLogin Interval	Specifies the time (in minutes) after which the Teamcenter services with log off from and log back on to Teamcenter.	Time in minutes.	1140
Service.DataSetOwner	Specifies the owner of visualization datasets created by the dispatcher client.	**DC** or **CAD**. **DC** means owner is DC Proxy User. **CAD** means owner is the same as the owner of the translation datasets.	

Property Name	Description	Valid values	Default values
Service.StoreJTFiles InSourceVol	Specifies that visualization data files for a source dataset will be stored in the same volume as the associated source dataset data files.	Boolean value.	**TRUE**
Service.UpdateExisting VisualizationData	Specifies that the dispatcher client should update existing visualization data for subsequent translations of the same version.	Boolean value.	**FALSE**
Service.PollingInterval	Specifies (in seconds) how long the translation process should sleep before it again queries for a dispatcher request.	Time in seconds.	
Service.CheckFor DuplicateRequests	Specifies whether validation processing should reject duplicate dispatcher requests.	Boolean value.	**TRUE**
Service.Tc. ConnectDelay	Specifies (in seconds) the Teamcenter server connection wait time between retries.	Time in seconds.	1

Dispatcher request cleanup properties — The request cleanup allows for automatic cleanup of completed dispatcher equest objects in the database and the staging directory.

Property name	Description	Valid values	Default values
Service.Request Cleanup.Successful. Interval	Specifies (in minutes) how often to query for and delete successful dispatcher request objects.	Time in minutes	
Service.Request Cleanup.Successful. Threshold	Specifies (in minutes) the amount of time that must have passed since a successful dispatcher request is last modified before it can be deleted.		
Service.Request Cleanup.UnSuccessful. Interval	Specifies (in minutes) how often to query for and delete unsuccessful dispatcher request objects.		
Service.Request Cleanup.UnSuccessful. Threshold	Specifies (in minutes) the amount of time that must have passed since an unsuccessful dispatcher request is last modified before it can be deleted.		

E-mail properties — The following are the recognized dispatcher client e-mail properties:

Property name	Description	Valid values	Default values
Service.Email. AdminEmailId	Specifies the adminstrator e-mail addresses to receive notification.	One or more valid, comma-delimited, e-mail addresses.	
Service.Email. AdminEmailId Delimiter	Specifies the delimiter used for the **Service.Email.AdminEmailId** property value.	A valid delimiter character.	**,**
Service.Email. SMTPServerName	Specifies the SMTP server.	A valid IP address of the SMTP Server	
Service.Email. SendUserEmail OnError	Specifies whether an e-mail should be sent when a translation error occurs.	**0**=off, **1**=on.	0
Service.Email. SenderId	Specifies the e-mail address that will send error notifications.	A single valid e-mail address.	The dispatcher client name.
Service.Email. EmailSubjectPrefix	Specifies the message to prefix to the e-mail subject.	String	[TS_ERROR]

Filter properties — You can control translation services using dispatcher client filters. Dispatcher client filters provide a convenient way to control dispatcher request objects that are exposed to the dispatcher client for processing. If filters are specified for a dispatcher client, all dispatcher requests must pass this filter, or the dispatcher request is not processed by the dispatcher client.

Property name	Description	Valid values	Default values
Service.Filters	Specifies a list of filters to apply to the dispatcher request object. For example: `Service.Filters=TranslatorFilter`	Comma-separated list of filters.	
Service.*Filter name***.** *Filter property*	Specifies the property value for a filter. For example:		

Property name	Description	Valid values	Default values
	`Service.TranslatorFilter.Provider=SIEMENS`		
Filter.*Filter name*	Specifies the class that implements the filter. For example: `Filter.TranslatorFilter=com.teamcenter.ets.request.TranslatorFilter`		

Translator properties — For more information about **Prepare**, **FileMap**, and **Load** classes, see the Java documentation in the **DispatcherClient/docs/dc/javadoc** directory.

Property name	Description	Valid values	Default values
Translator.*Provider name***.***Translator name***.Prepare**	Specifies the name of the class used to extract data from Teamcenter, which will be provided to the translator. For example: `Translator.SIEMENS.atob.Prepare=com.teamcenter.ets.translator.ugs.atob.TaskPrep`		
Translator.*Provider name***.***Translator name***.FileMap**	Specifies the name of the class used to match the files produced by the translator to the source data.		
Translator.*Provider name***.***Translator name***.Load**	Specifies the name of the class used to save the results of the translation in theTeamcenter database and result files in the Teamcenter volume. For example: `Translator.SIEMENS.atob.Load=com.teamcenter.ets.translator.ugs.atob.DatabaseOperation`		

Property name	Description	Valid values	Default values
Translator.*Provider name*.*Translator name*.**Duplicate**	Specifies whether a translator-specific duplicate check should be turned on or not.	Boolean	
Translator.*Provider name*.*Translator name*.**Log**	Specifies the log files to be attached to the dispatcher request.	Comma-delimited log file names.	
Translator.*Provider name*.*Translator name*.**LogsForComplete**	Specifies whether log files should be attached to dispatcher request if translation is successful.	Boolean	

Siemens Industry Software

Headquarters
Granite Park One
5800 Granite Parkway
Suite 600
Plano, TX 75024
USA
+1 972 987 3000

Americas
Granite Park One
5800 Granite Parkway
Suite 600
Plano, TX 75024
USA
+1 314 264 8499

Europe
Stephenson House
Sir William Siemens Square
Frimley, Camberley
Surrey, GU16 8QD
+44 (0) 1276 413200

Asia-Pacific
Suites 4301-4302, 43/F
AIA Kowloon Tower, Landmark East
100 How Ming Street
Kwun Tong, Kowloon
Hong Kong
+852 2230 3308

About Siemens PLM Software

Siemens PLM Software is a leading global provider of product lifecycle management (PLM) software and services with 7 million licensed seats and 71,000 customers worldwide. Headquartered in Plano, Texas, Siemens PLM Software works collaboratively with companies to deliver open solutions that help them turn more ideas into successful products. For more information on Siemens PLM Software products and services, visit www.siemens.com/plm.